project submission date: 27/10/2026

NAME: SRIKANTH CH
MOBILE NO: 8618279920
BATCH: AWS BATCH 1
PROJECT NAME: AWS Peering connection. Private Communication Between Two VPCs
region: Mumbai/app-south-1a

1. Project Overview

This project demonstrates how to establish private network communication between two Amazon Virtual Private Clouds (VPCs) using **AWS VPC Peering**.

I created two VPCs in the AWS Mumbai region (ap-south-1), configured their subnets and route tables, launched three EC2 instances, and established a VPC peering connection. Initially, the EC2 instances could not communicate across the VPCs. After accepting the peering request, configuring routes, and updating security group rules, I tested connectivity using the ping command.

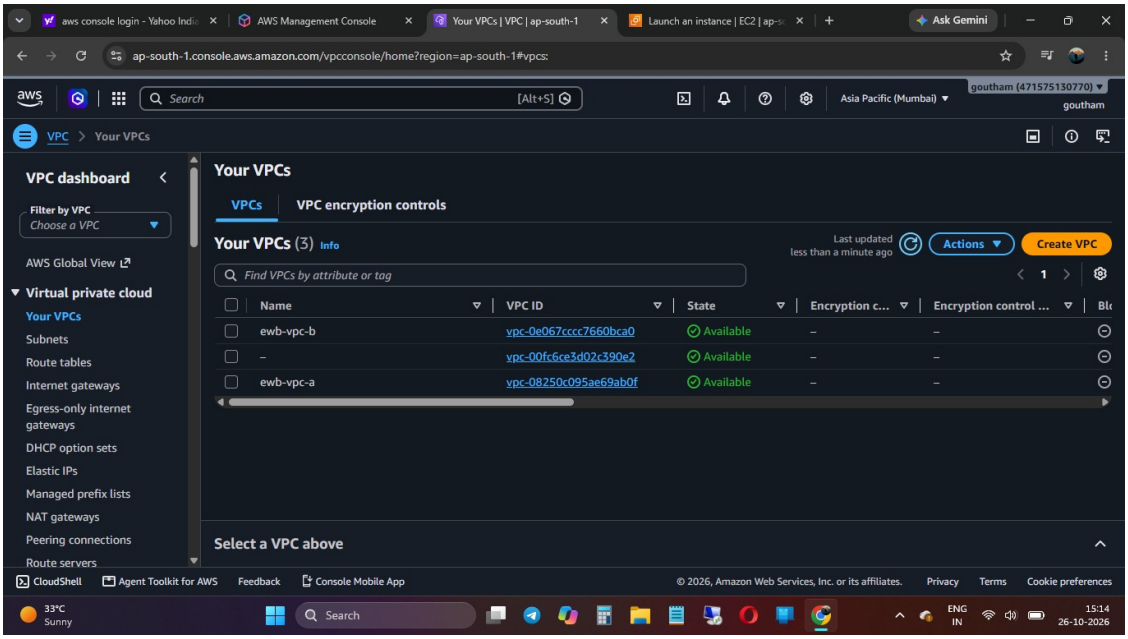
Project Objectives

- Create two isolated VPCs with different CIDR blocks.
- Configure public and private subnets.
- Create route tables and configure subnet associations.
- Launch EC2 instances in different subnets.
- Establish a VPC peering connection.
- Configure routing and security group rules.
- Test private communication between EC2 instances.

AWS Services Used

- Amazon VPC
- Amazon EC2
- VPC Peering
- Route Tables
- Internet Gateway
- Security Groups
- Amazon Linux

Step 1: Create the First VPC – ewb-vpc-a



I created the first VPC and named it ewb-vpc-a.

Configuration:

VPC Name: ewb-vpc-a
IPv4 CIDR Block: 10.10.0.0/16
Region: Asia Pacific (Mumbai)
Availability Zone for subnets: ap-south-1a
Tenancy: Default

The CIDR block 10.10.0.0/16 provides a private IP address range for the resources inside this VPC.

2: Create the Second VPC – ewb-vpc-b

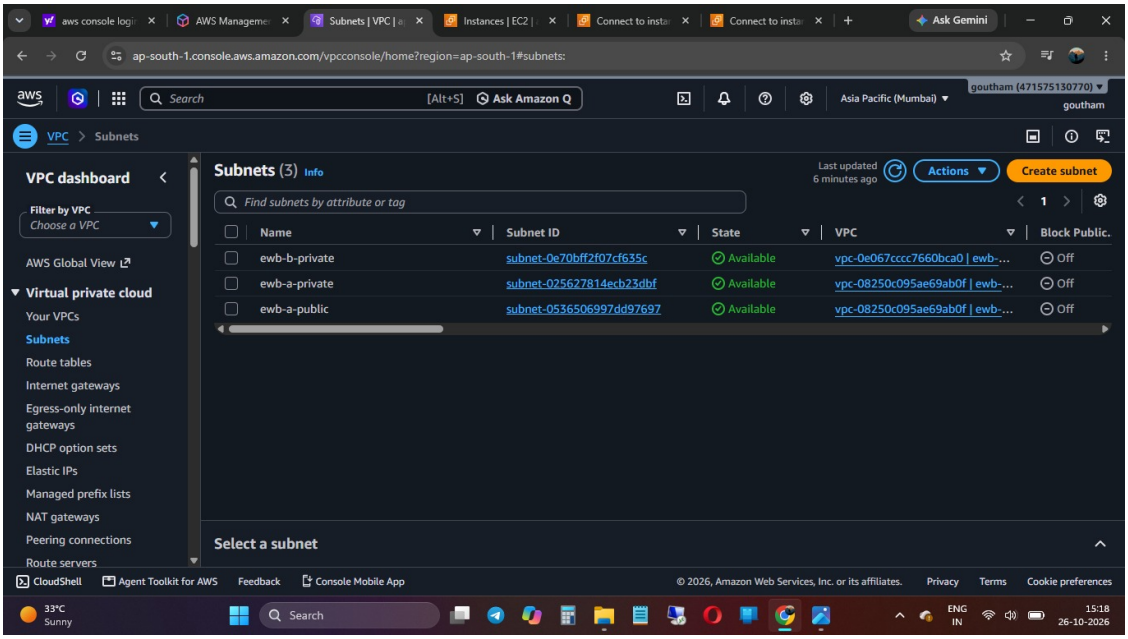
Next, I created another VPC named ewb-vpc-b with a different CIDR block.

Configuration:

VPC Name: ewb-vpc-b
IPv4 CIDR Block: 10.20.0.0/16
Region: Asia Pacific (Mumbai)
Tenancy: Default

I used a different CIDR range to avoid overlapping IP addresses between the two VPCs. Non-overlapping CIDR blocks are essential for this VPC peering design.

Step 2: Create Subnets in Both VPCs



I created three subnets to organize the network resources.

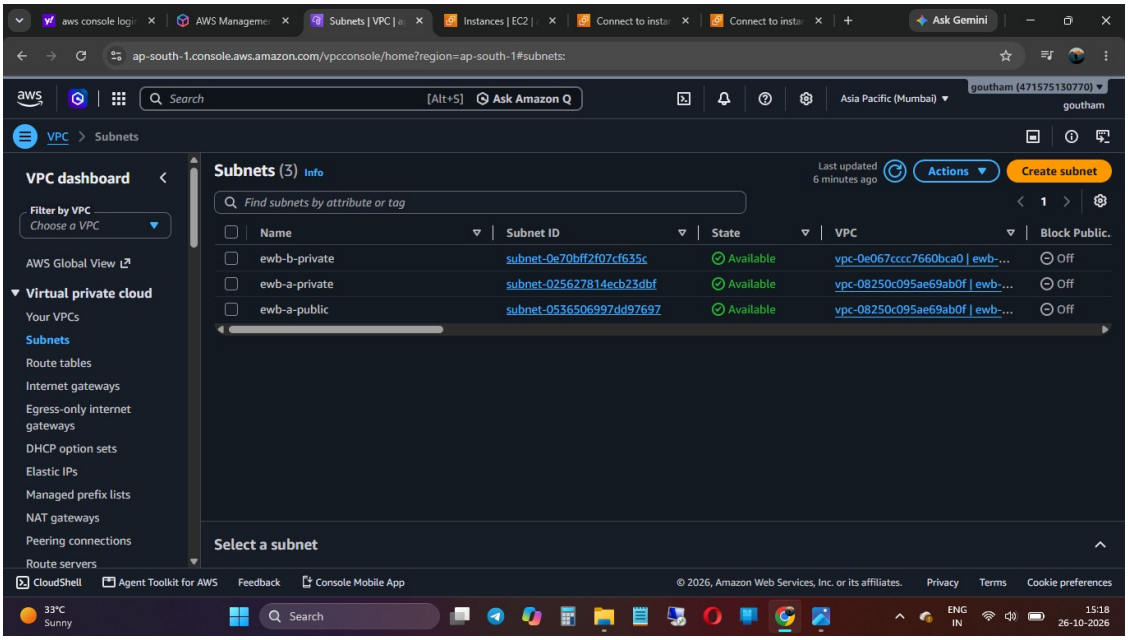
1. Public subnet in ewb-vpc-a

I created a public subnet named ewb-a-public.

Subnet Name: ewb-a-public
VPC: ewb-vpc-a
CIDR Block: 10.10.1.0/24
Availability Zone: ap-south-1a

This subnet is intended to host the bastion EC2 instance. A subnet is considered public when its route table provides a route to an Internet Gateway; an instance also needs a public IP and suitable security rules for direct internet access.

2. Private subnet in ewb-vpc-a

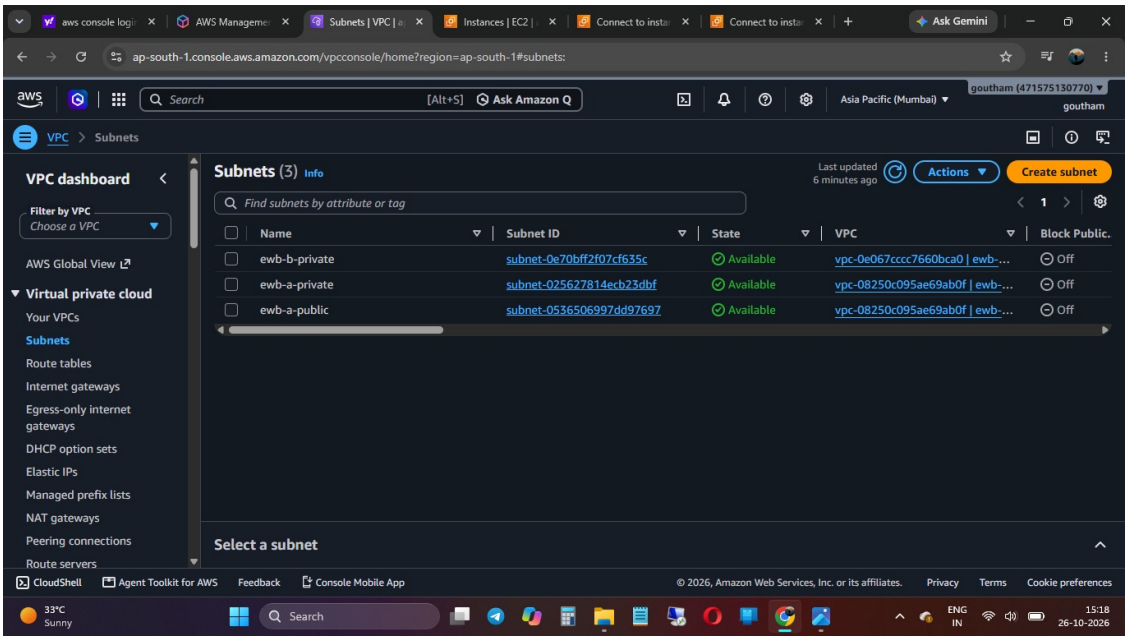


I created a private subnet named ewb-a-private.

Subnet Name: ewb-a-private
VPC: ewb-vpc-a
CIDR Block: 10.10.2.0/24
Availability Zone: ap-south-1a

This subnet hosts the ewb-app-a EC2 instance. It is intended for private communication without requiring a public IP address.

3. Private subnet in ewb-vpc-b



I created another private subnet named ewb-b-private.

Subnet Name: ewb-b-private

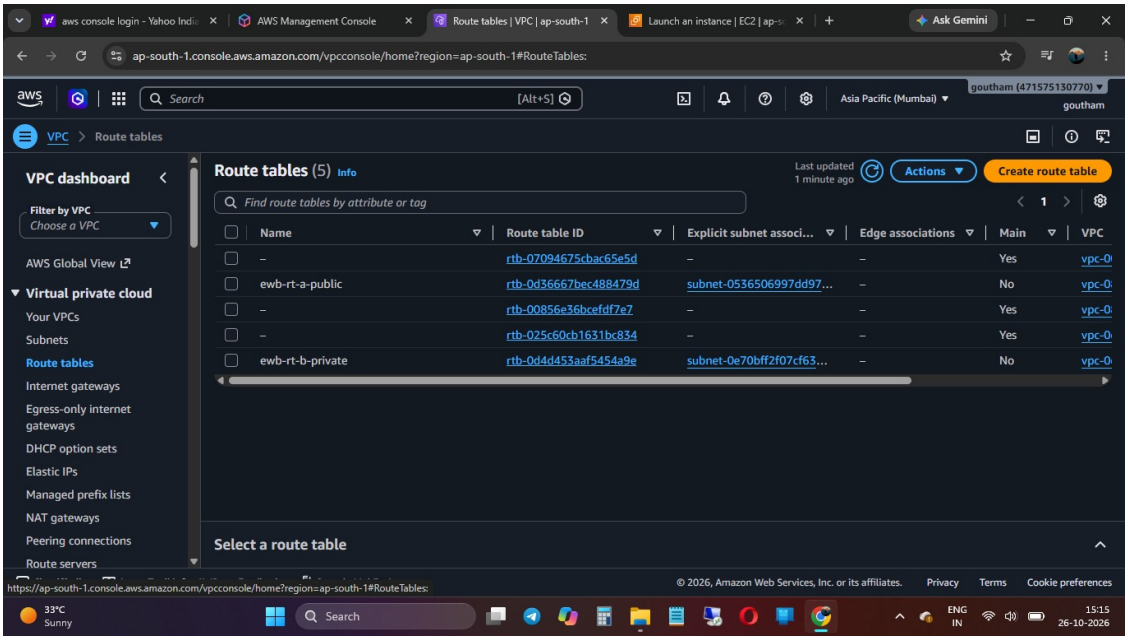
VPC: ewb-vpc-b
CIDR Block: 10.20.1.0/24
Availability Zone: ap-south-1a

This subnet hosts the ewb-app-b EC2 instance. It will communicate with resources in the first VPC through the VPC peering connection.

Step 3: Configure Route Tables

Route tables determine where network traffic is sent. I configured routing for internet access from the public subnet and prepared routing for communication between the two VPCs.

1. Create the public route table



I created a route table named ewb-rt-a-public and associated it with ewb-vpc-a.

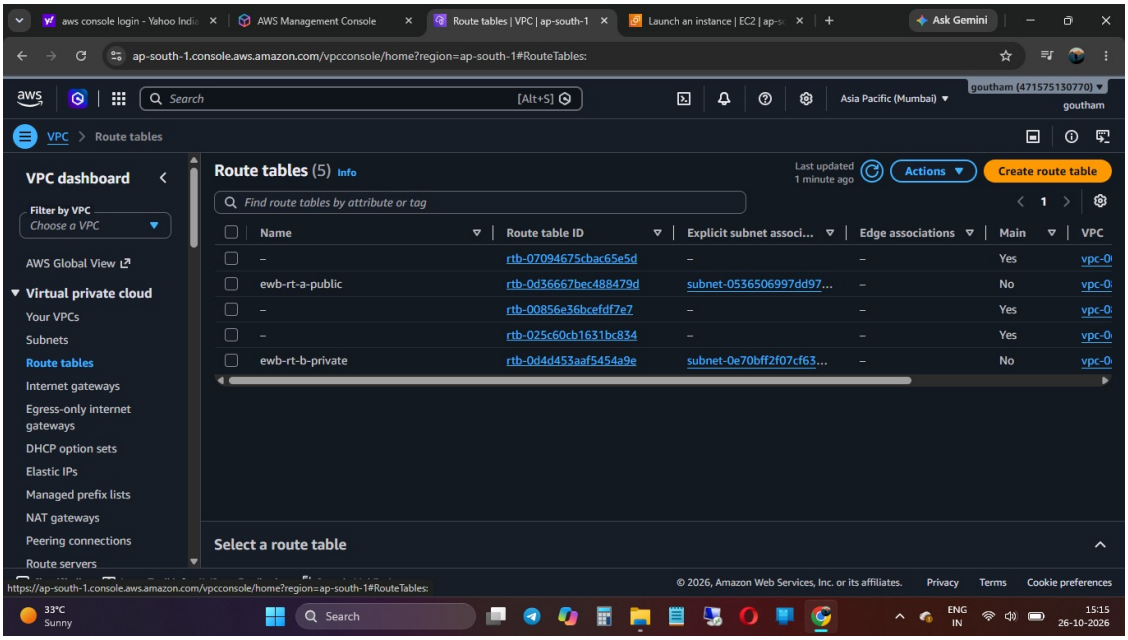
I explicitly associated the ewb-a-public subnet with this route table.

I then added the following route:

Destination: 10.10.0.0/16, Target: Local
Destination: 0.0.0.0/0, Target: ewb-igw-a Internet Gateway

The local route is automatically created by AWS. The 0.0.0.0/0 route directs internet-bound traffic through the Internet Gateway.

2. Configure the private route table for ewb-vpc-b



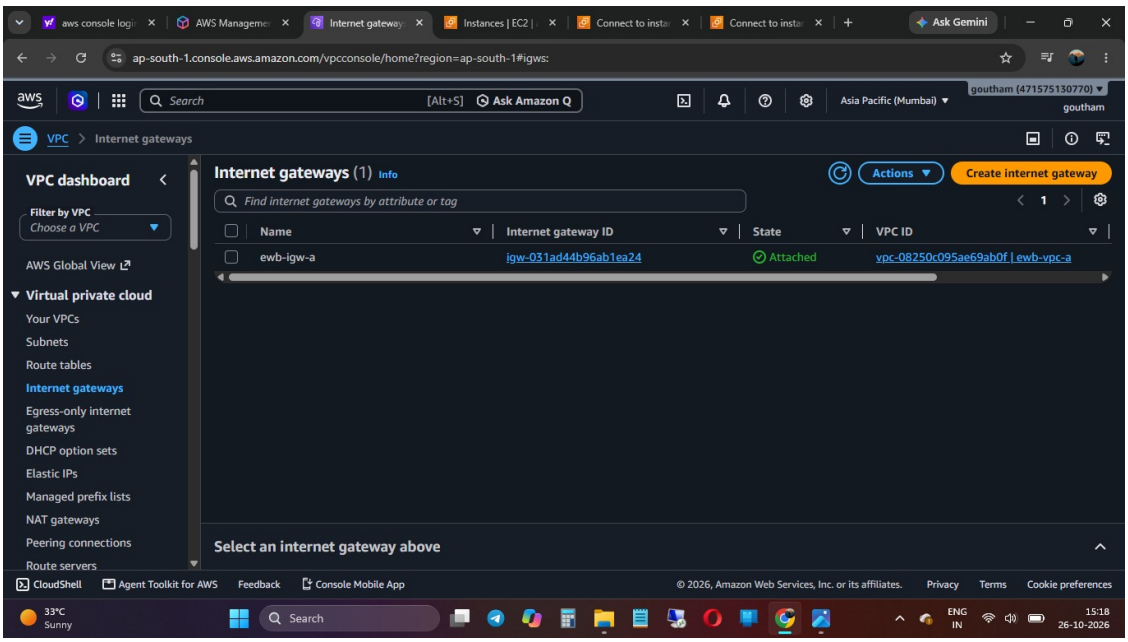
I created a route table named ewb-rt-b-private and associated it with ewb-vpc-b.

I explicitly associated the ewb-b-private subnet with this route table.

Initially, I did not configure an Internet Gateway route for this private subnet because the goal was to communicate with the other VPC privately.

Important: The route tables must also contain routes to the remote VPC CIDRs after peering is established. Ensure that the route table associated with ewb-a-private also has the appropriate peering route. Do not assume the public route table automatically controls the private subnet.

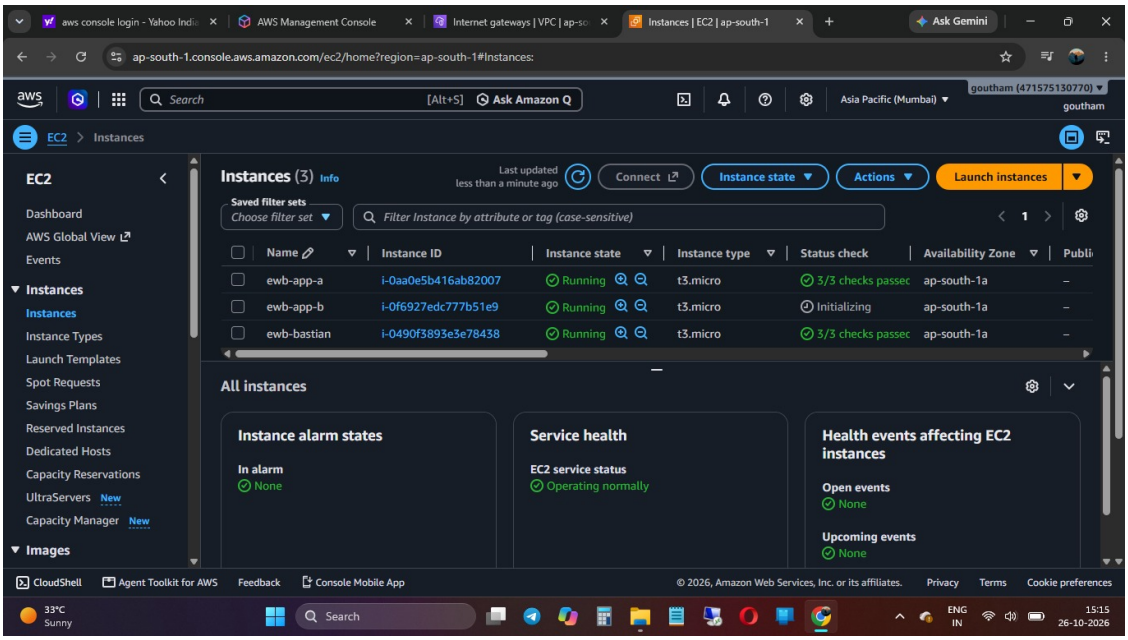
STEP 4: add internet gateway for vpc-a public subnet



Step 5: Launch Three EC2 Instances

I launched three Amazon Linux EC2 instances in the three subnets.

1. Launch the bastion instance



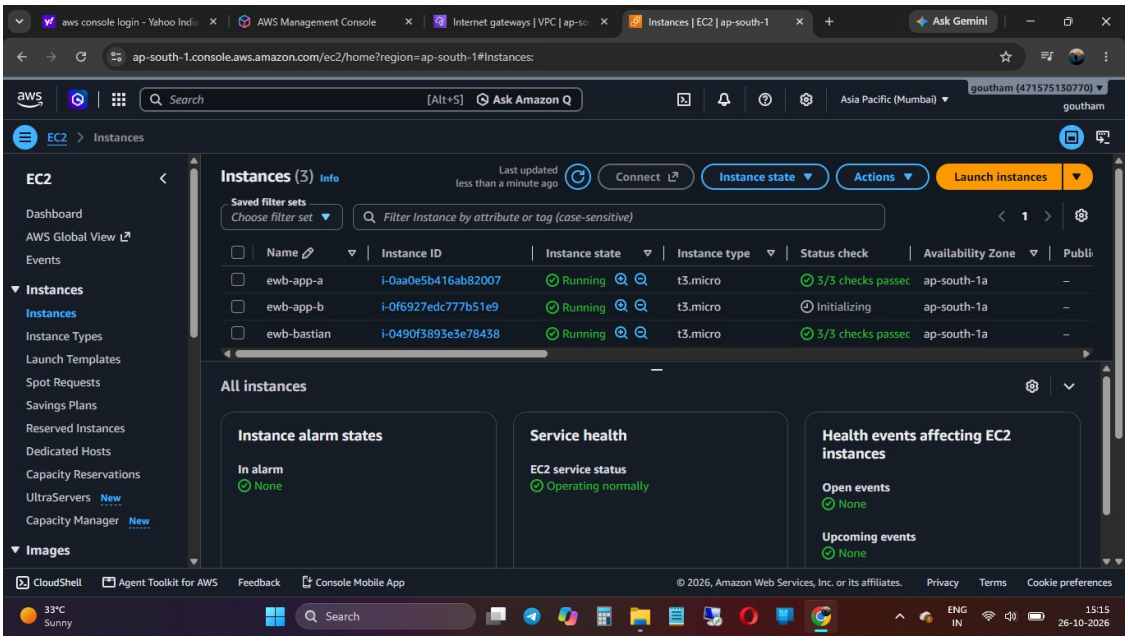
Instance name: ewb-bastian

I selected Amazon Linux, created a key pair named ewb-lab, and downloaded the private key file ewb-lab.pem.

Under Network Settings, I selected ewb-vpc-a and the ewb-a-public subnet. I enabled automatic public IP assignment.

This instance is intended to act as a bastion host for administrative access to the private environment.

2. Launch the application instance in VPC A



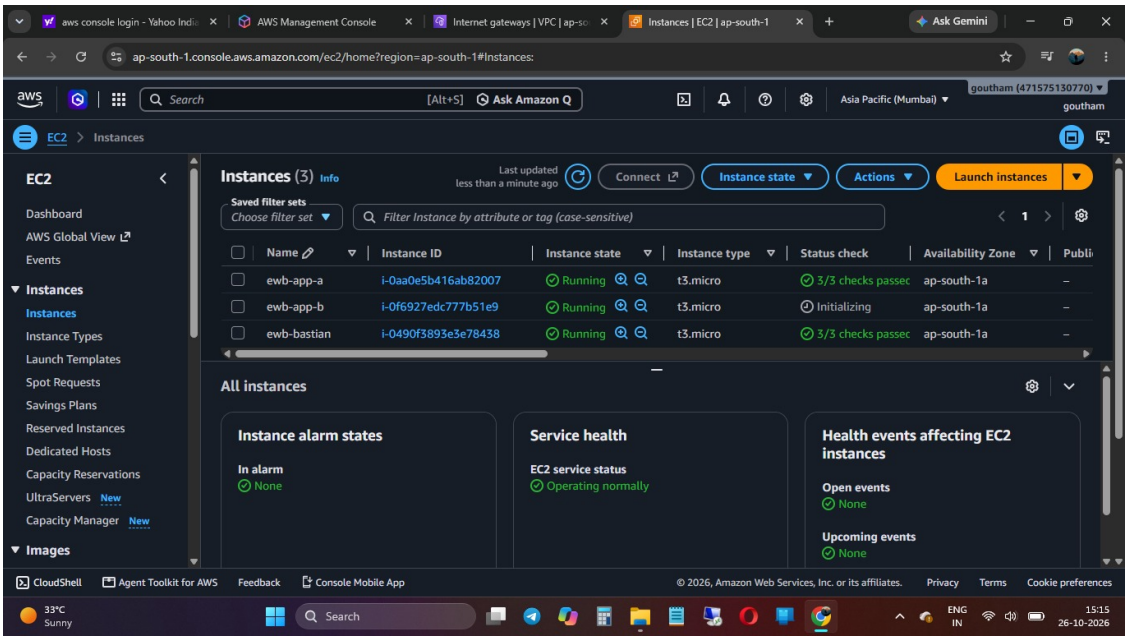
Instance name: ewb-app-a

I launched the second EC2 instance in the ewb-a-private subnet.

VPC: ewb-vpc-a
Subnet: ewb-a-private
CIDR: 10.10.2.0/24
Key Pair: ewb-lab
Auto-assign Public IP: Disabled

This instance is designed to operate without a public IP address and communicate with other resources through private networking.

3. Launch the application instance in VPC B



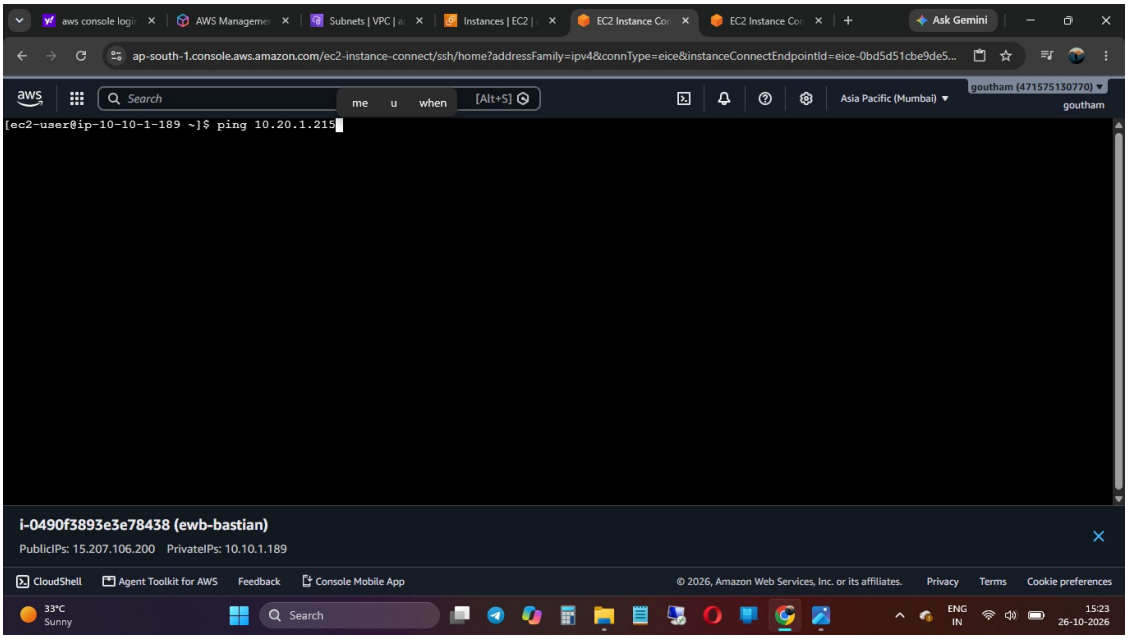
Instance name: ewb-app-b

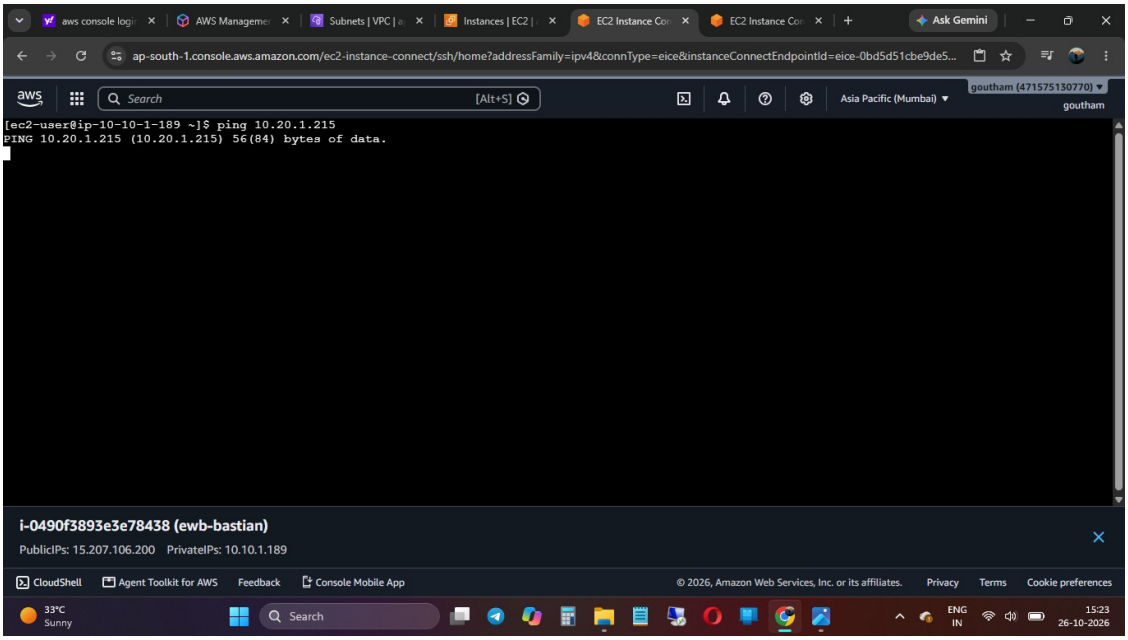
I launched the third EC2 instance in the ewb-b-private subnet.

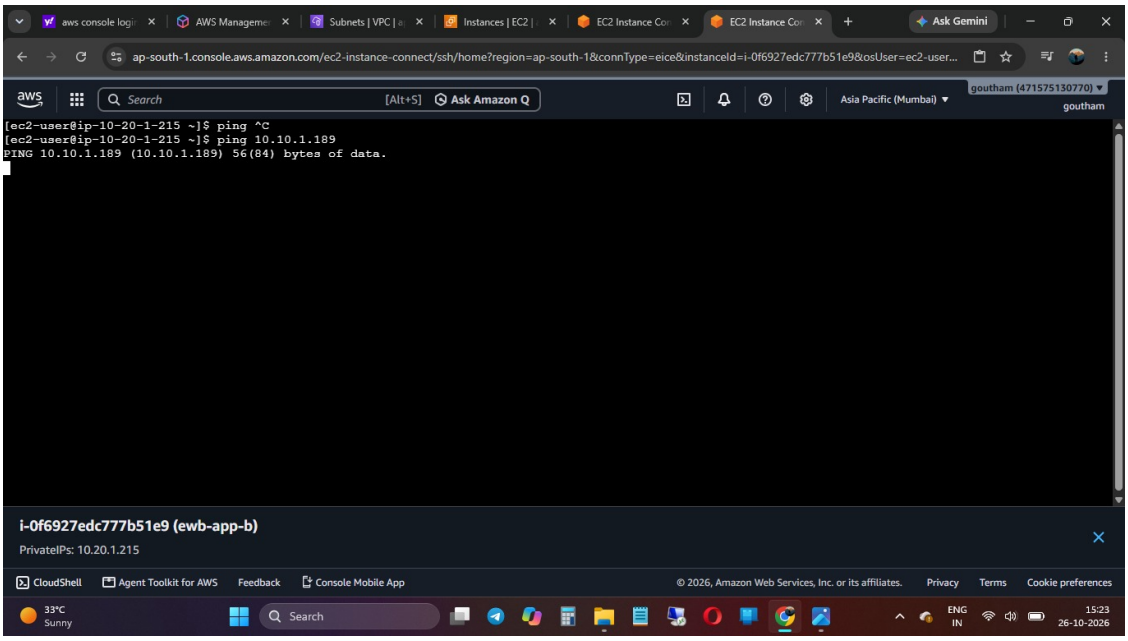
VPC: ewb-vpc-b
Subnet: ewb-b-private
CIDR: 10.20.1.0/24
Key Pair: ewb-lab
Auto-assign Public IP: Disabled
Security Group: ewb-sg-app-b

This instance will communicate with ewb-app-a through the VPC peering connection.

STEP 6: Test Initial Connectivity







After launching the EC2 instances, I attempted to test communication between the application instances using the ping command.

At this stage, the two VPCs did not have a configured peering connection and the required cross-VPC routes were not in place. Therefore, private communication was not yet working.

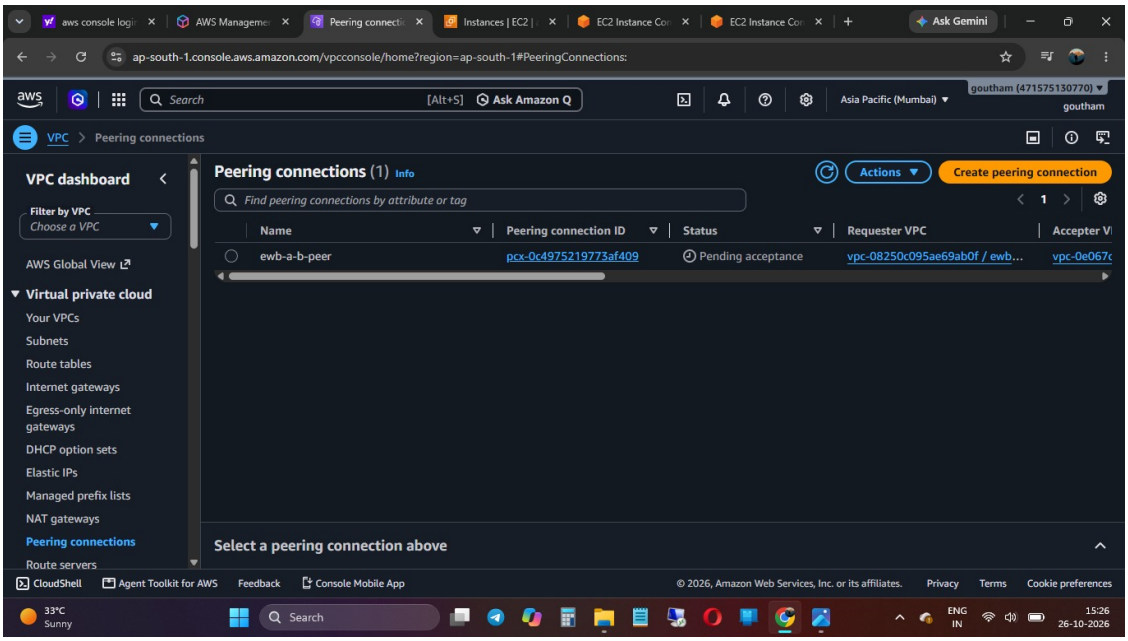
Example command, run from ewb-app-a using the private IP address of ewb-app-b:

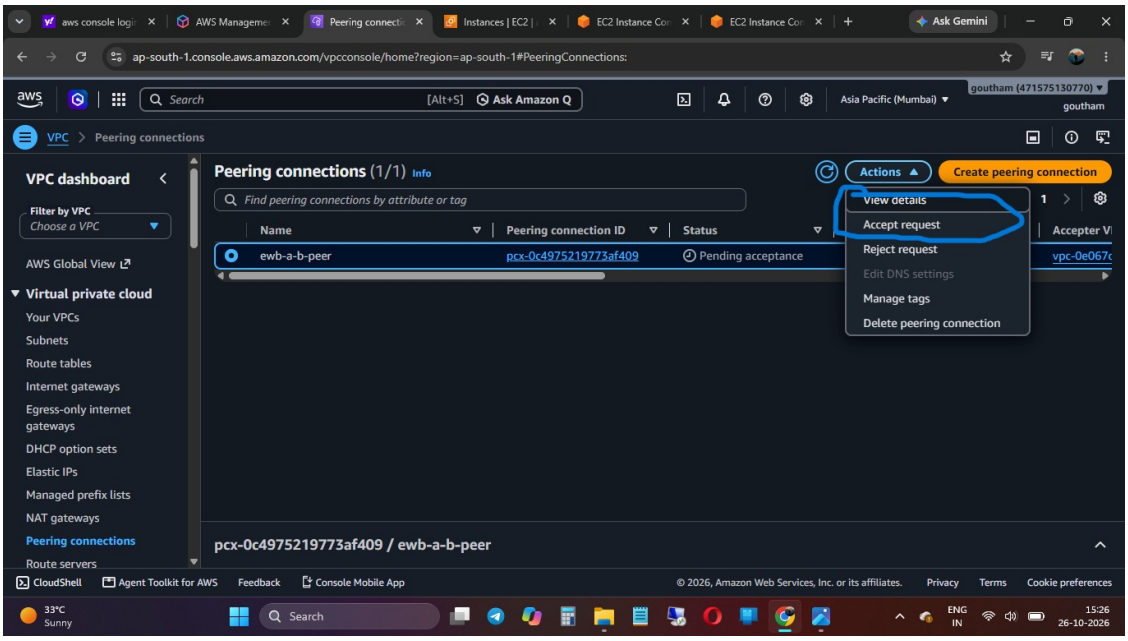
```
ping 10.20.1.215
```

Similarly, from ewb-app-b, I attempted to ping the private IP address of ewb-app-a.

The tests did not receive replies. This demonstrated that creating VPCs and launching EC2 instances alone does not automatically enable communication between separate VPCs.

STEP 7: Create the VPC Peering Connection





To enable private communication, I created a VPC peering connection named ewb-a-b-peer.

Configuration:

Peering Connection Name: ewb-a-b-peer
Requester VPC: ewb-vpc-a
Accepter VPC: ewb-vpc-b
AWS Account: Same account
Region: Same region — Mumbai

The peering connection provides a private networking path between the two VPCs. However, creating the request alone does not complete the connection.

Accept the peering request

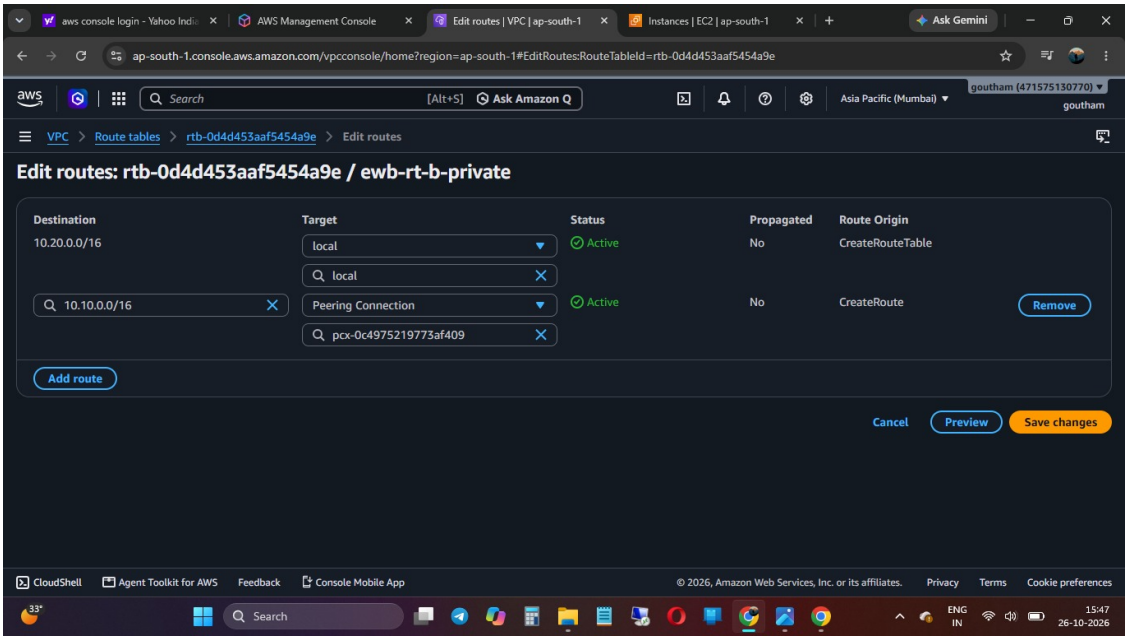
I selected the newly created peering connection and opened the Actions menu. I then selected **Accept request** to approve the connection from the accepter VPC.

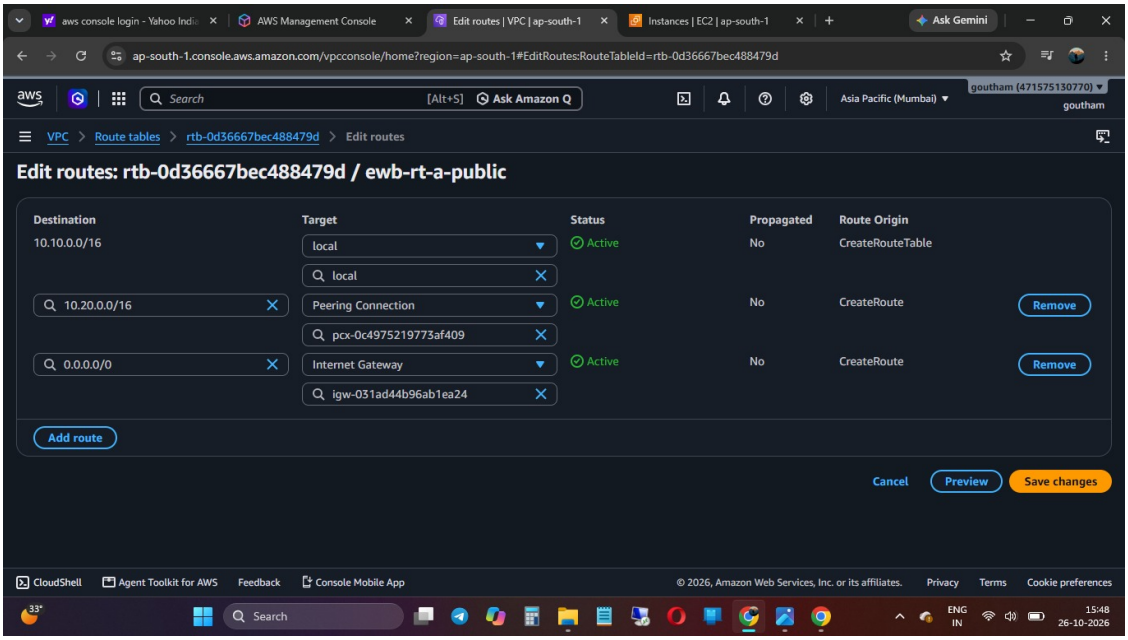
After acceptance, I verified that the peering connection status showed Active.

STEP 8: Add Routes for VPC Peering

A peering connection does not automatically update the route tables. I needed to add routes to both sides so that traffic could reach the remote VPC.

1. Add the route in VPC A





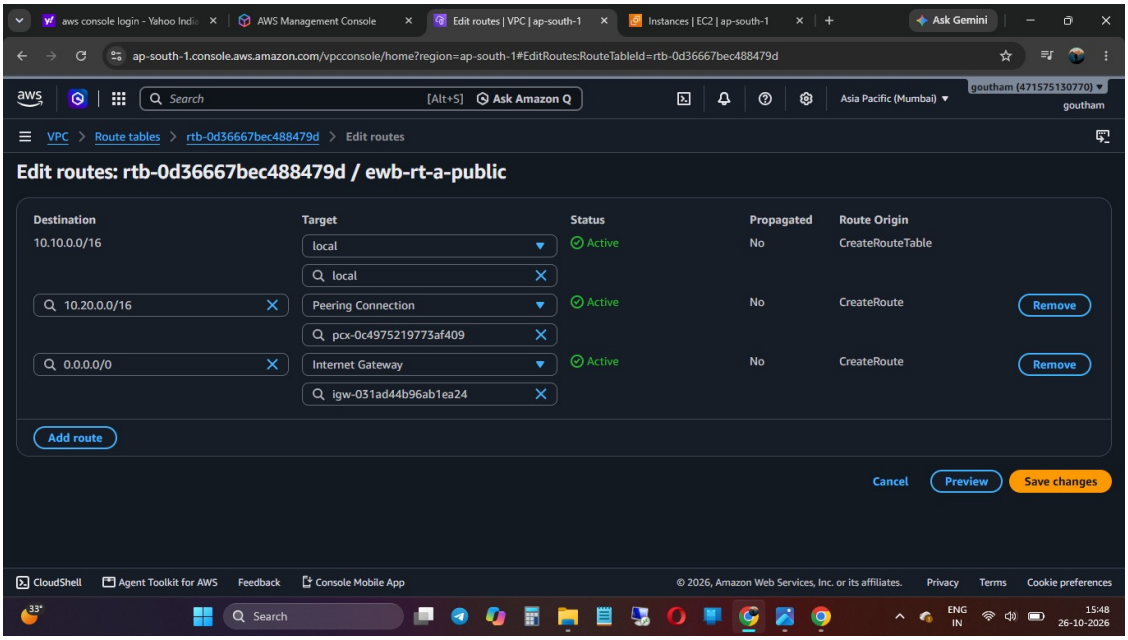
I edited the route table associated with the ewb-a-private subnet and added the following route:

Destination: 10.20.0.0/16
Target: ewb-a-b-peer

This route directs traffic from the private subnet in VPC A toward VPC B through the peering connection.

If the subnet uses the main route table rather than a separately named private route table, I must add the route to that actual associated route table.

2. Add the route in VPC B



I edited ewb-rt-b-private and added the following route:

Destination: 10.10.0.0/16
Target: ewb-a-b-peer

This route directs traffic from VPC B toward VPC A through the same peering connection.

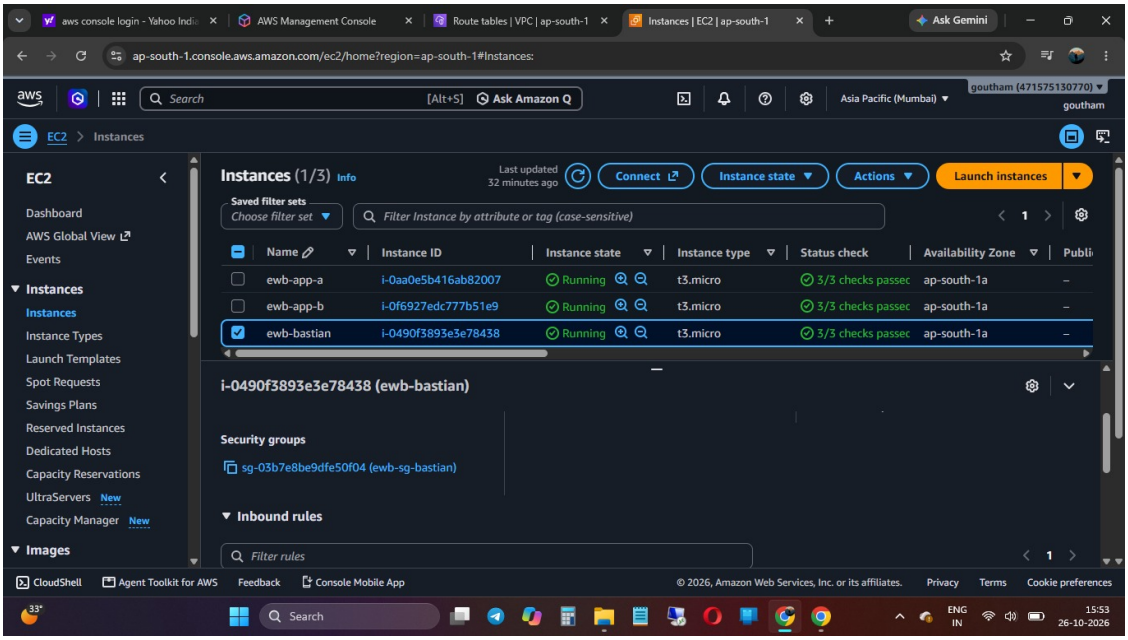
These two routes establish the routing path in both directions. Security group rules must still allow the traffic..

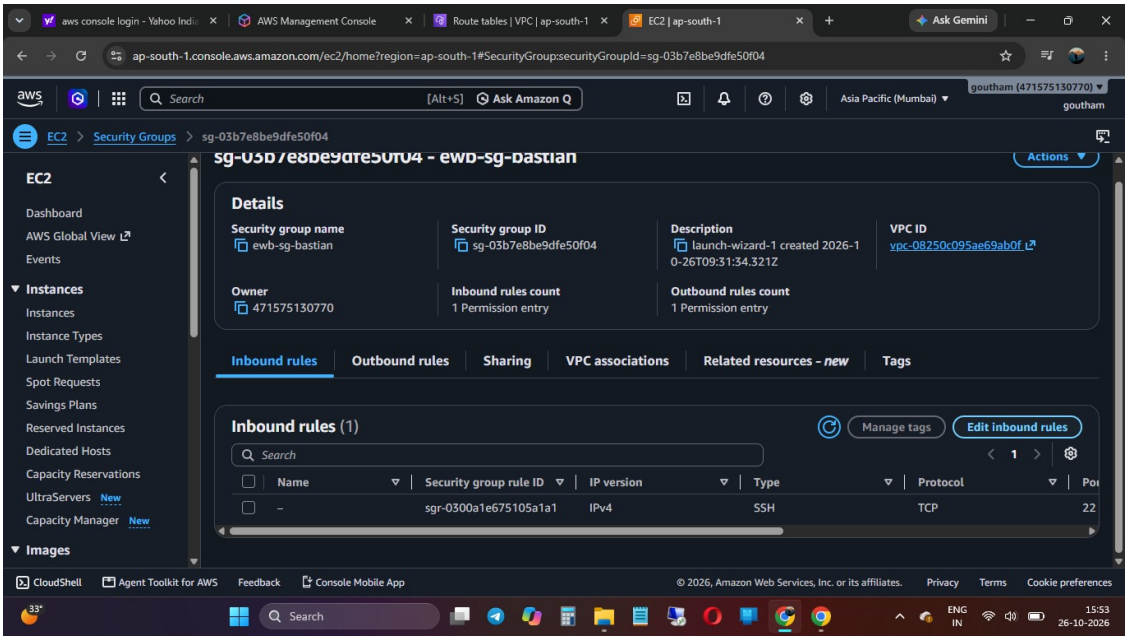
STEP 9: Configure Security Groups to Allow Ping

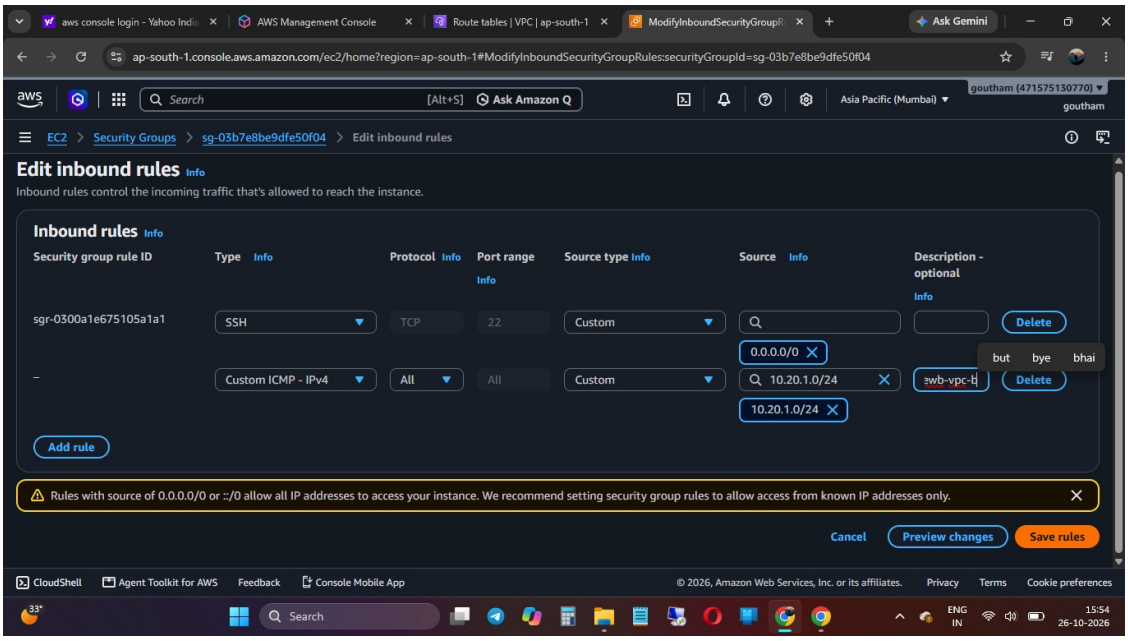
Even after the peering connection became active and the routes were added, the EC2 instances might not respond to ping requests if their security groups block ICMP traffic.

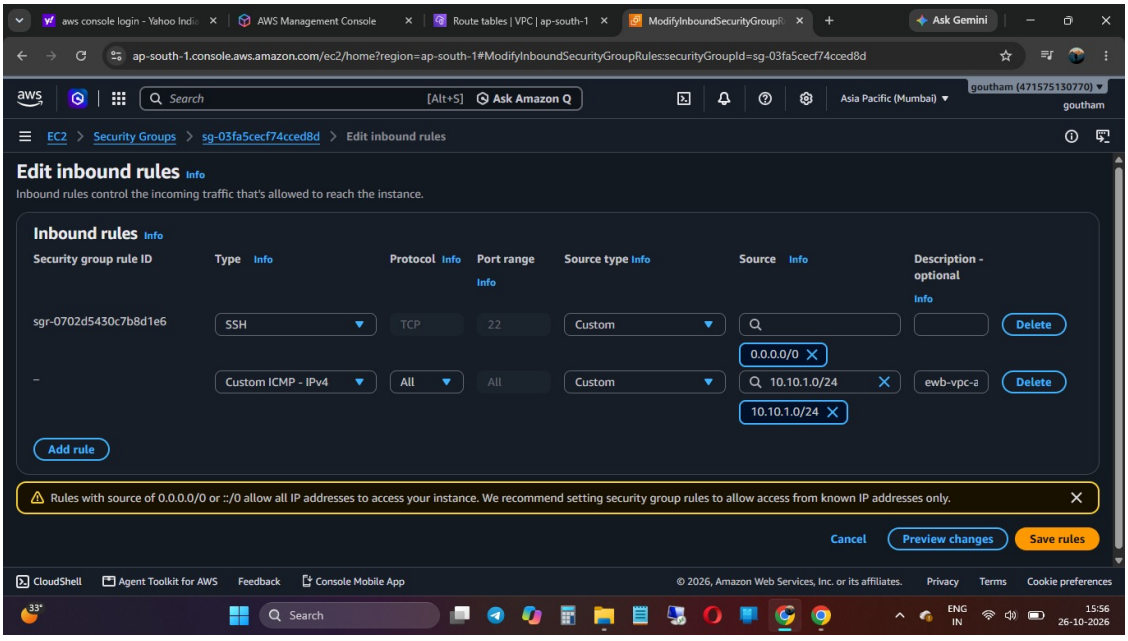
I updated the inbound rules of the application instances to allow ICMP IPv4 traffic from the remote application subnet.

1. Configure the security group for ewb-app-a









I selected the security group attached to ewb-app-a, opened **Inbound rules**, and selected **Edit inbound rules**.

I added the following rule:

Type: All ICMP - IPv4
Source: 10.20.1.0/24
Purpose: Allow ping requests from VPC B's application subnet

This allows ICMP traffic originating from the ewb-b-private subnet.

2. Configure the security group for ewb-app-b

I opened the inbound rules of ewb-sg-app-b and added the following rule:

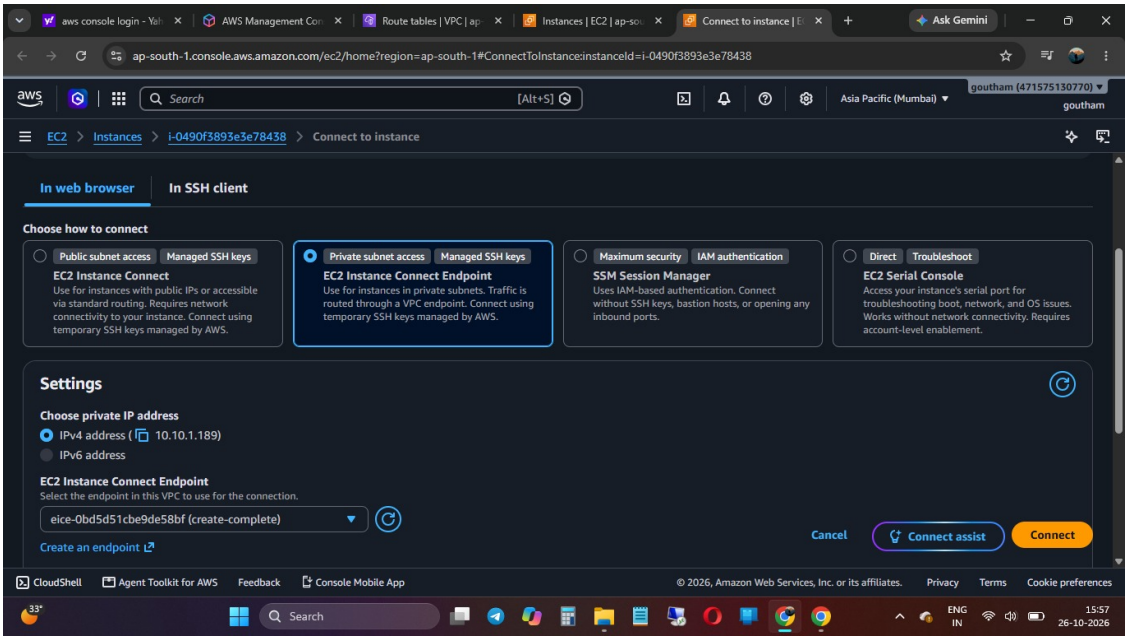
Type: All ICMP - IPv4
Source: 10.10.2.0/24

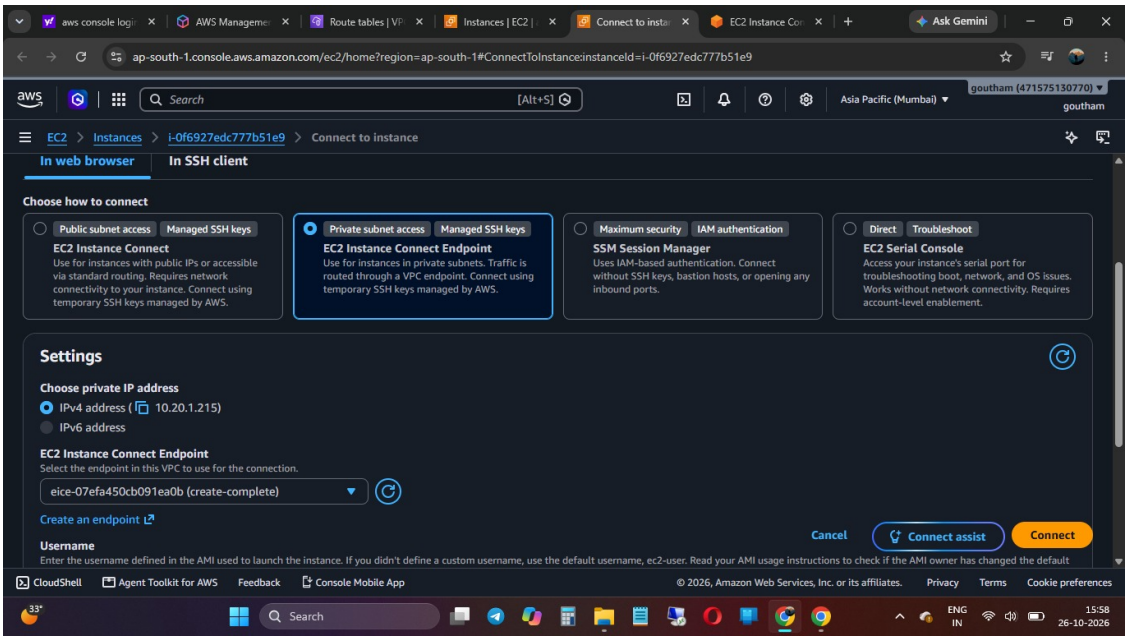
Purpose: Allow ping requests from VPC A's application subnet

Important CIDR correction: The ewb-app-a instance is in 10.10.2.0/24, not 10.10.1.0/24. Therefore, use 10.10.2.0/24 as the source for this rule when restricting access to the application subnet.

Security groups are stateful, so reply traffic for an allowed request is permitted automatically. The default outbound rules usually allow all outbound traffic unless they have been changed. Network ACLs must also permit the required traffic if custom ACL rules are configured.

STEP 10: Verify Connectivity Between the EC2 Instances



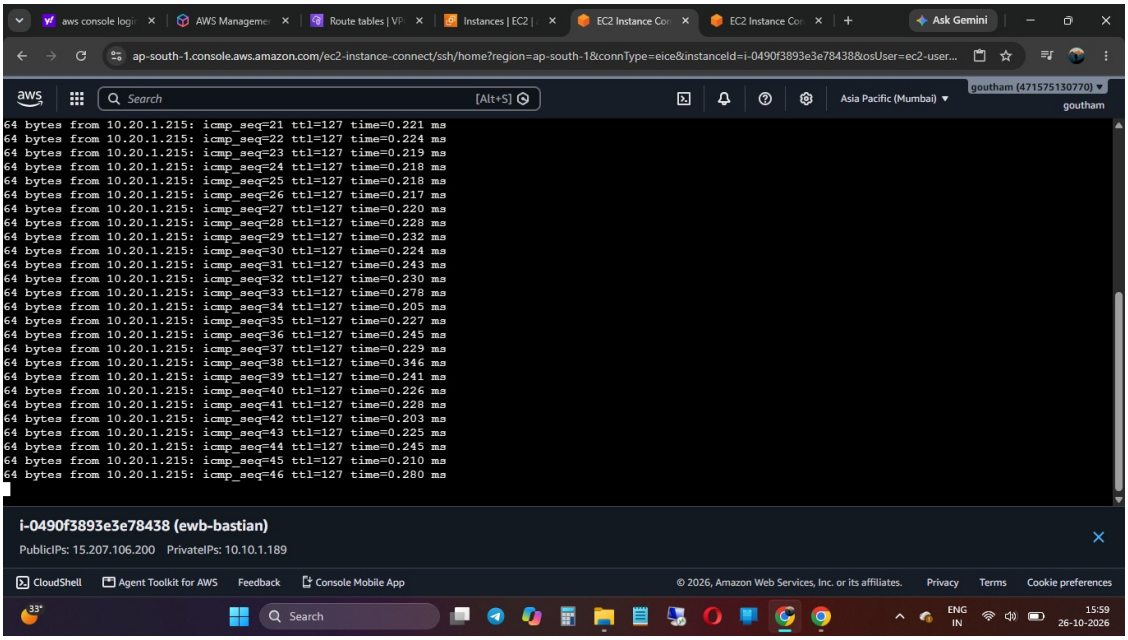


After accepting the peering request, configuring the routes, and updating the security groups, I tested connectivity again using the private IP addresses of the application instances.

Test from ewb-app-a to ewb-app-b

I connected to ewb-app-a through the bastion host or another permitted administrative access method and ran:

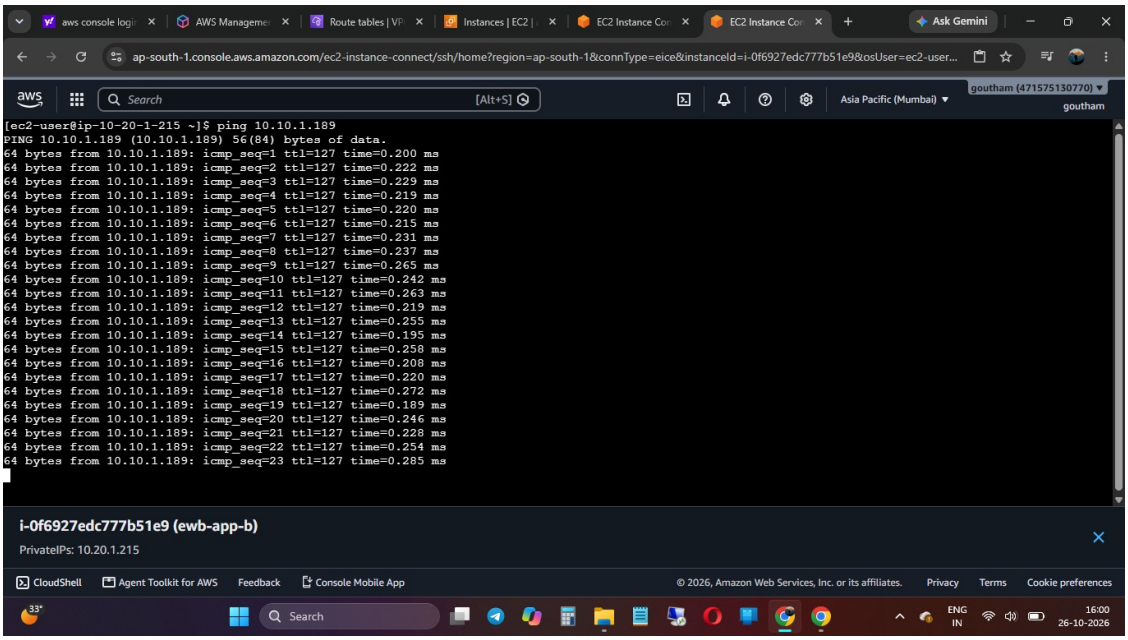
ping 10.20.1.215



Test from ewb-app-b to ewb-app-a

I also tested communication in the reverse direction:

ping 10.10.1.189



When the routes, security groups, and network ACLs are correctly configured and the operating system permits ICMP, the ping command should receive replies from the remote private IP address.

This confirms that the instances can communicate over the private network without requiring public IP addresses on the application instances.

STEP 11: Final Network Architecture

The final network design contains two VPCs connected through a VPC peering connection.

VPC A: ewb-vpc-a, CIDR 10.10.0.0/16
Public subnet 10.10.1.0/24: ewb-bastian
Private subnet 10.10.2.0/24: ewb-app-a

Connected through VPC Peering: ewb-a-b-peer

VPC B: ewb-vpc-b, CIDR 10.20.0.0/16
Private subnet 10.20.1.0/24: ewb-app-b

STEP 12: Project Outcome

Through this project, I learned how to:

1. Create and configure VPCs with separate IPv4 CIDR blocks.

2. Design public and private subnets.

3. Configure route tables and subnet associations.

4. Launch EC2 instances in different subnets.

5. Create and accept a VPC peering connection.

6. Configure routes for private communication between VPCs.

7. Troubleshoot network connectivity using security group rules.

8. Validate connectivity using the ping command and private IP addresses.

The project demonstrates how AWS VPC Peering can connect two VPCs privately when routing and security controls are configured correctly.

Conclusion

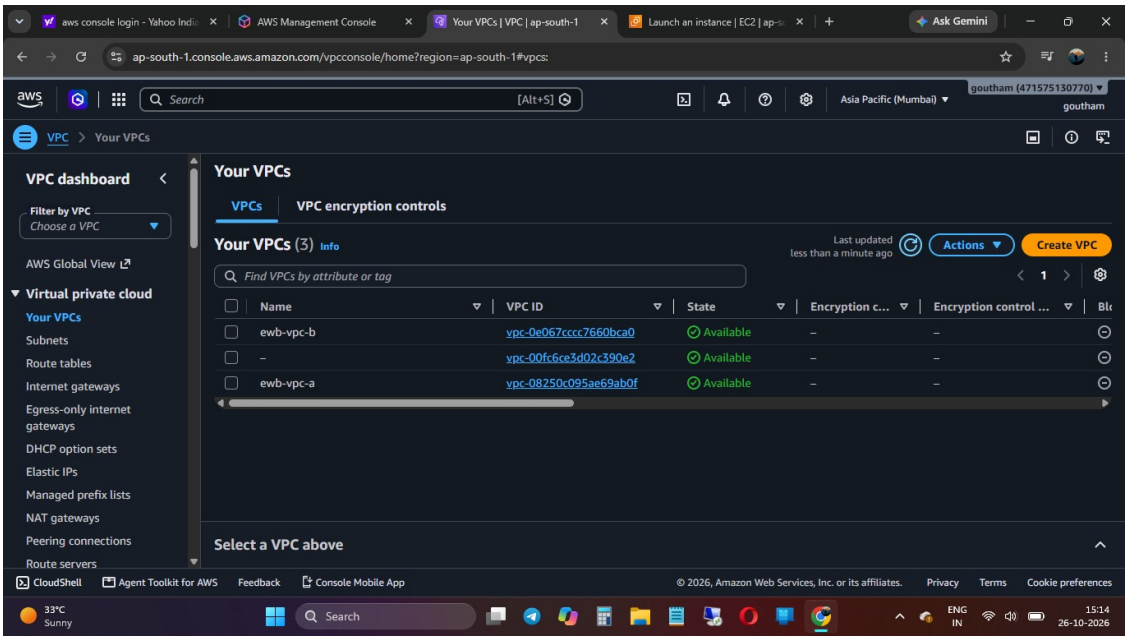
This internship project provided practical experience with AWS networking fundamentals. I learned that a successful VPC peering setup requires more than creating the peering connection: both VPCs need appropriate routes, and the relevant security controls must permit the intended traffic.

The final objective was to achieve private communication between ewb-app-a and ewb-app-b using their private IP addresses.

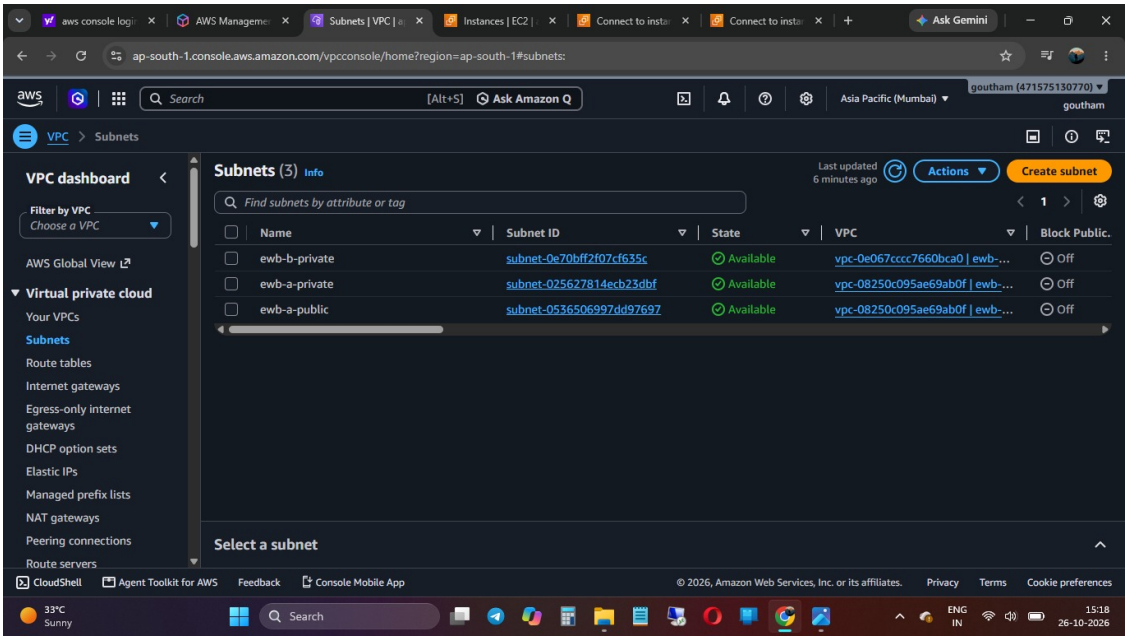
- Screenshots Checklist

Before publishing the README, make sure you have added these screenshots:

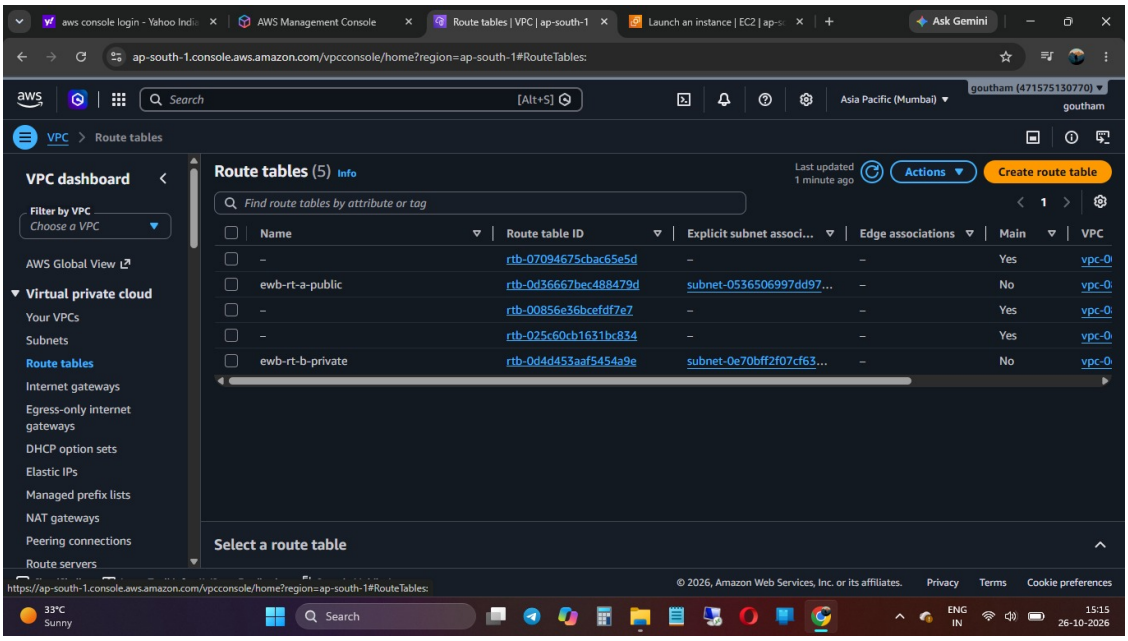
- Screenshot 1 – ewb-vpc-a -ewb-vpc-b



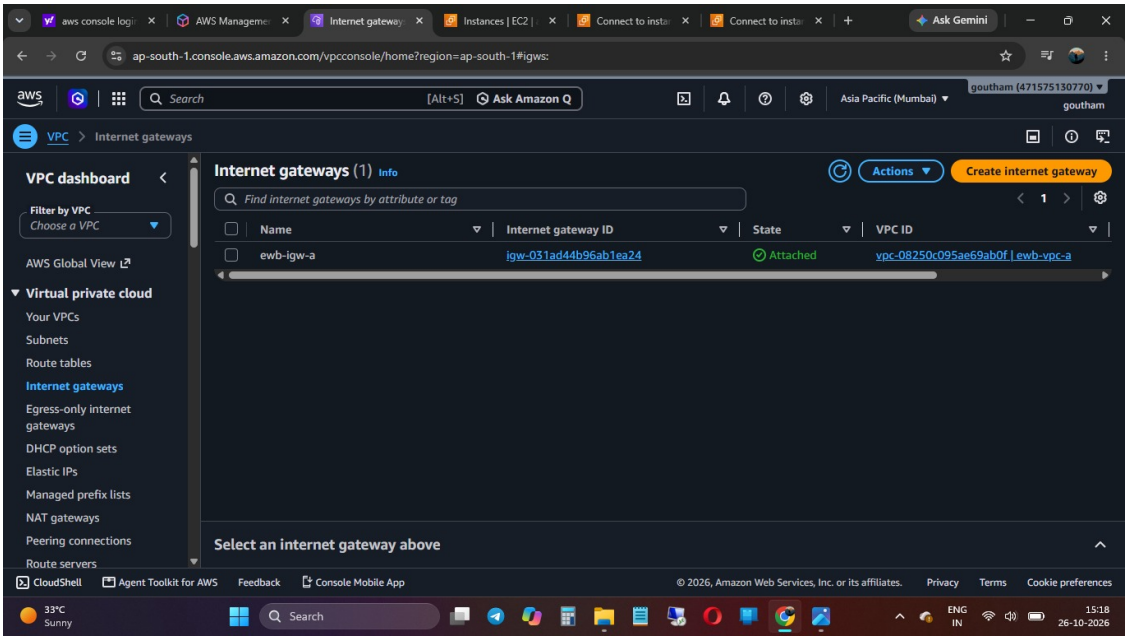
- Screenshot 2 – ewb-a-public - ewb-a-private - ewb-b-private



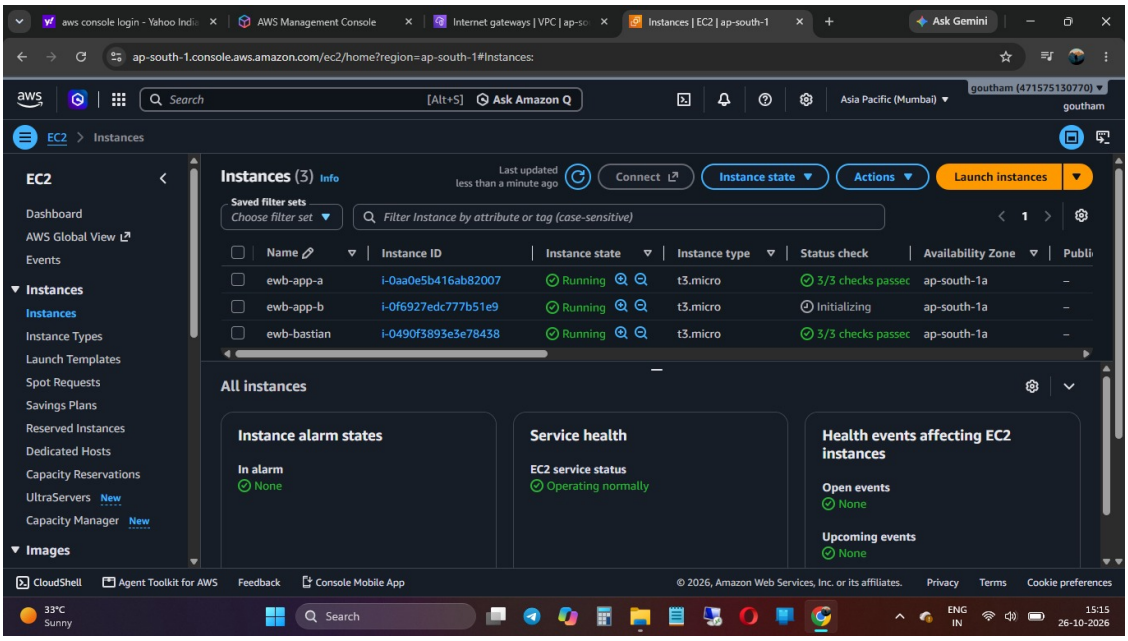
- Screenshot 3 – Public route table and subnet association - Private route table and subnet association



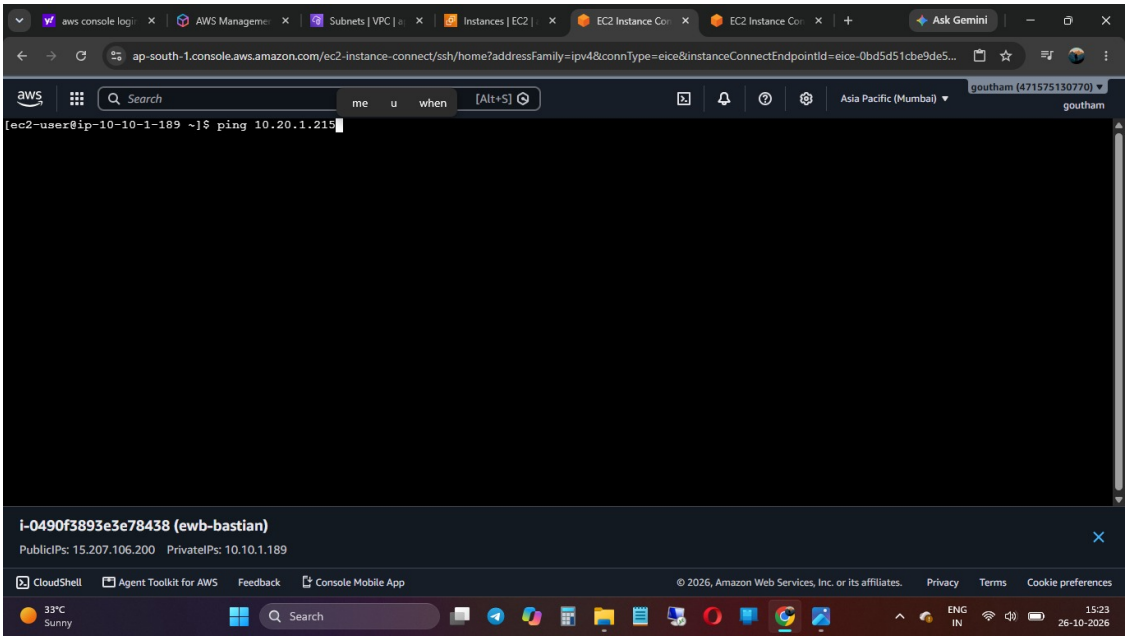
- Screenshot 4– Internet Gateway route

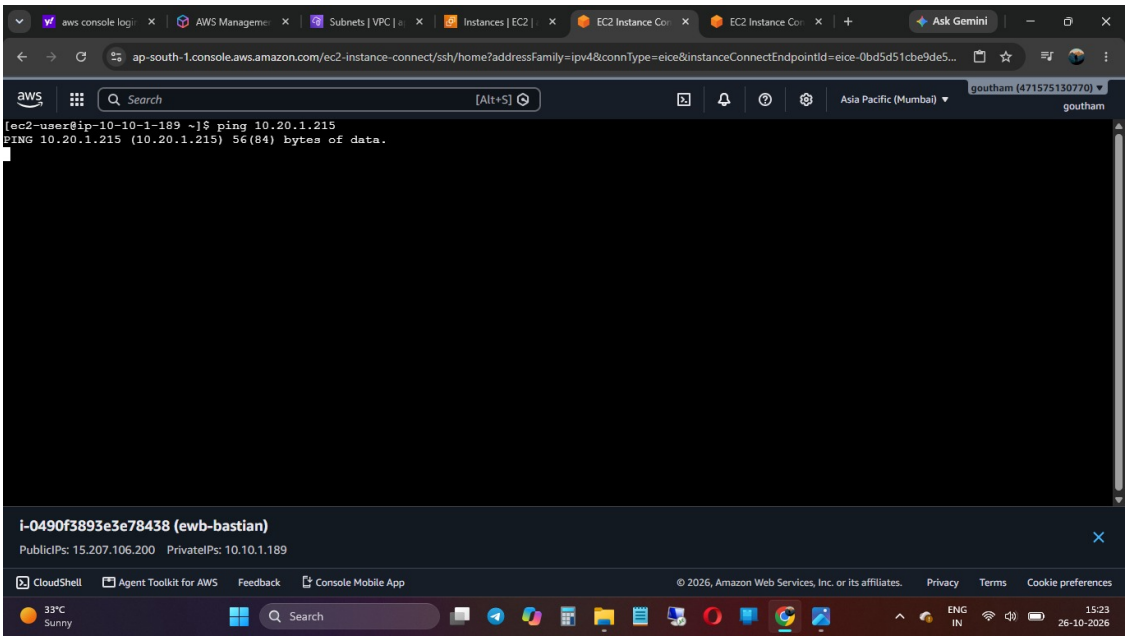


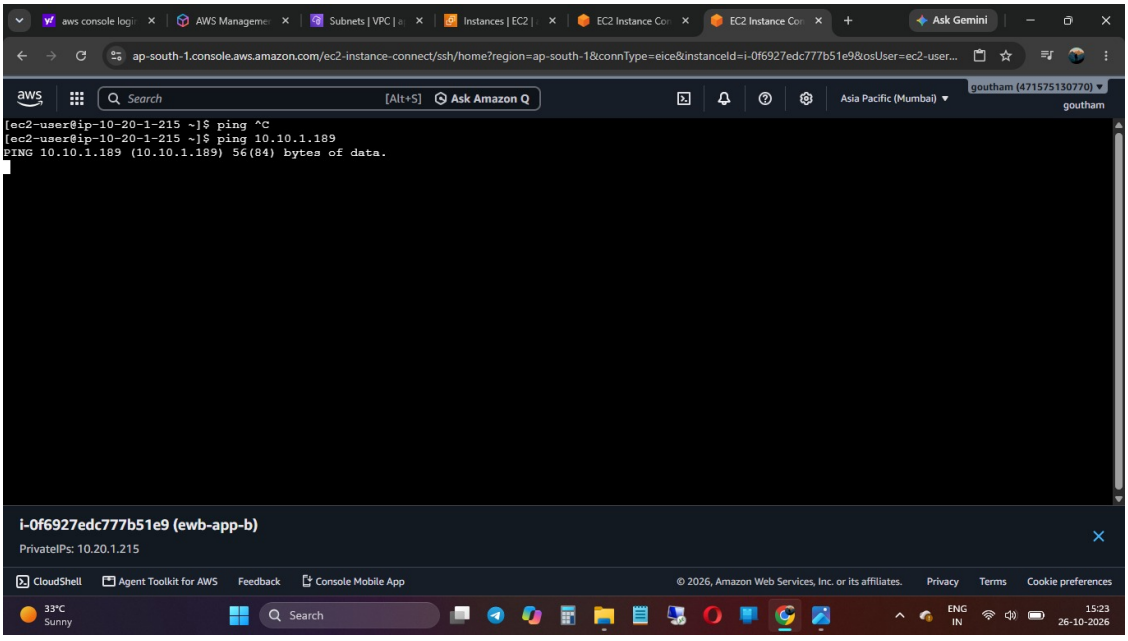
- Screenshot 5 – Bastion EC2 configuration - ewb-app-a configuration - ewb-app-b configuration



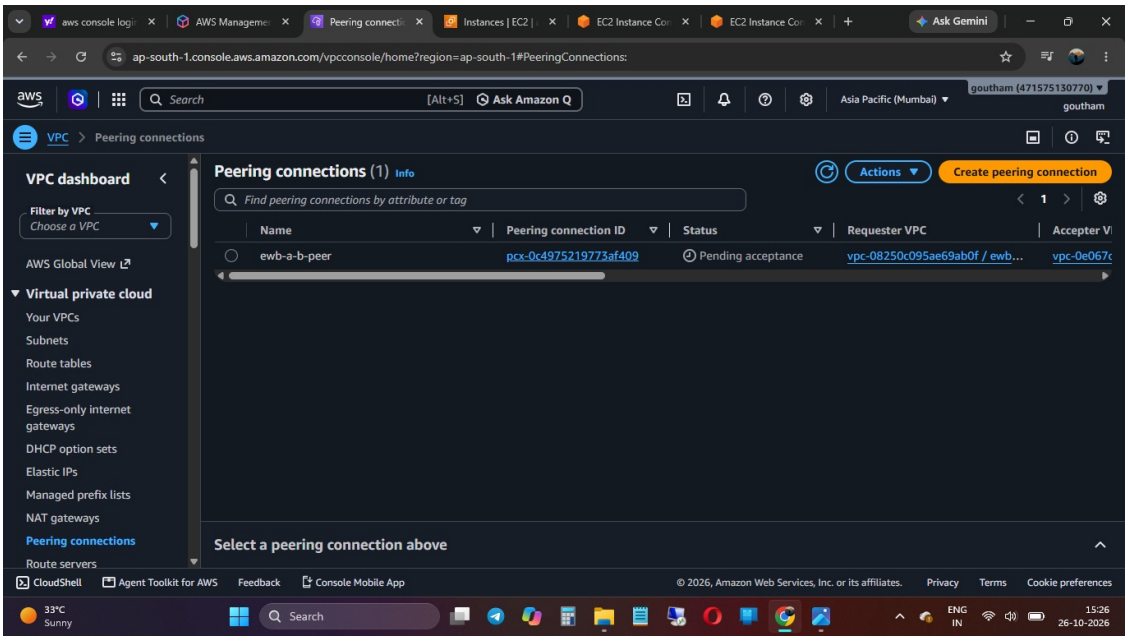
- Screenshot 6 – Initial failed ping test



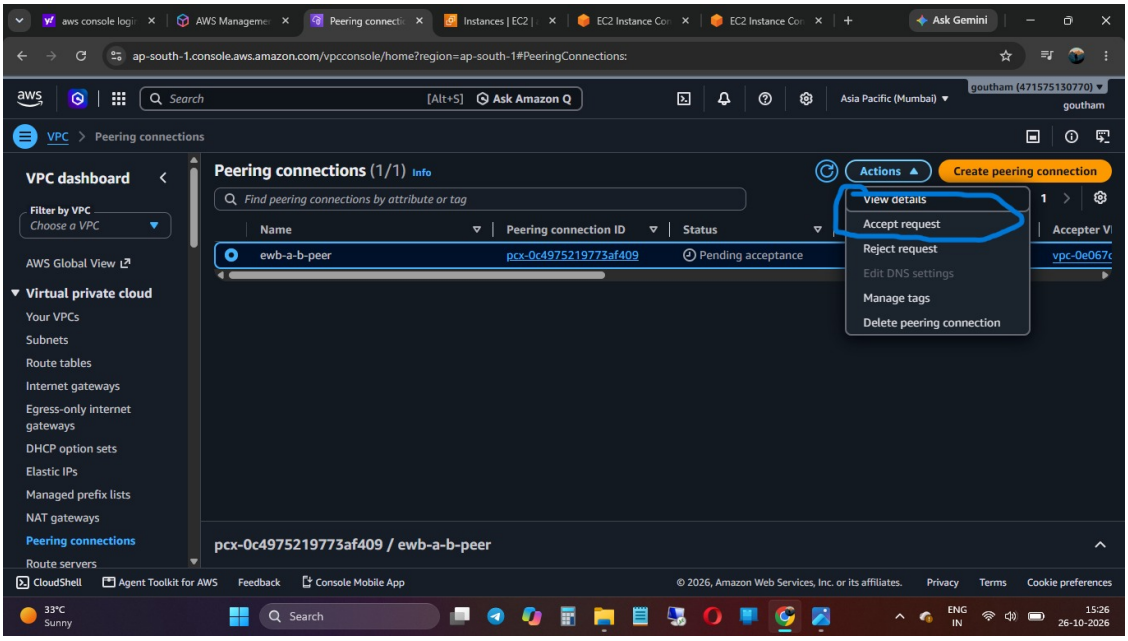




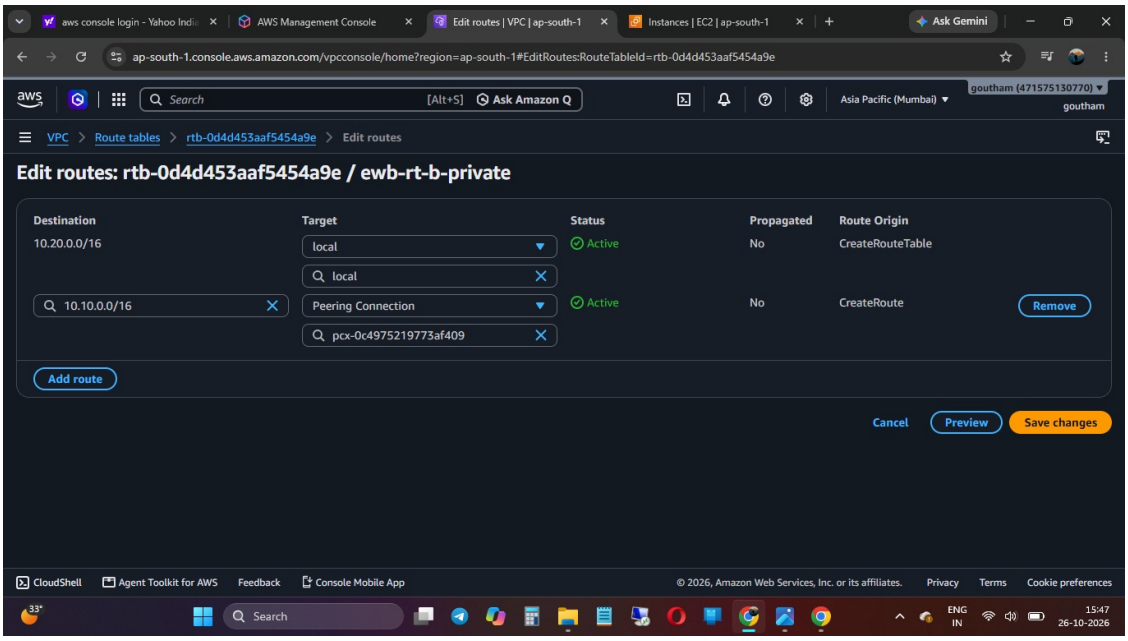
- Screenshot 7 – Active peering connection

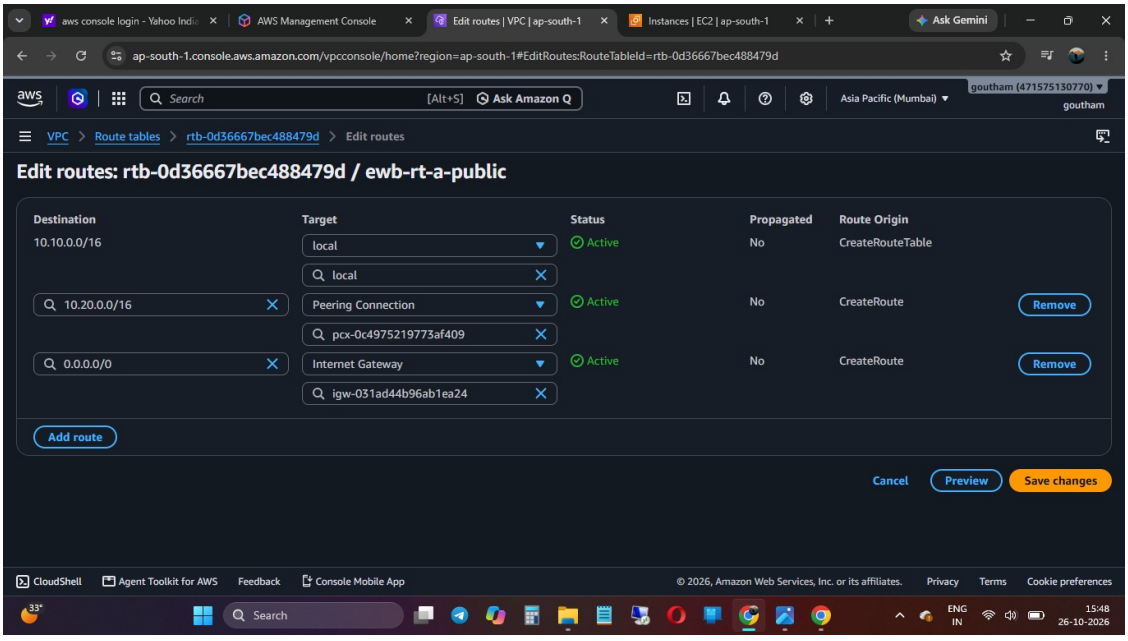


- Screenshot 8 – Peering request

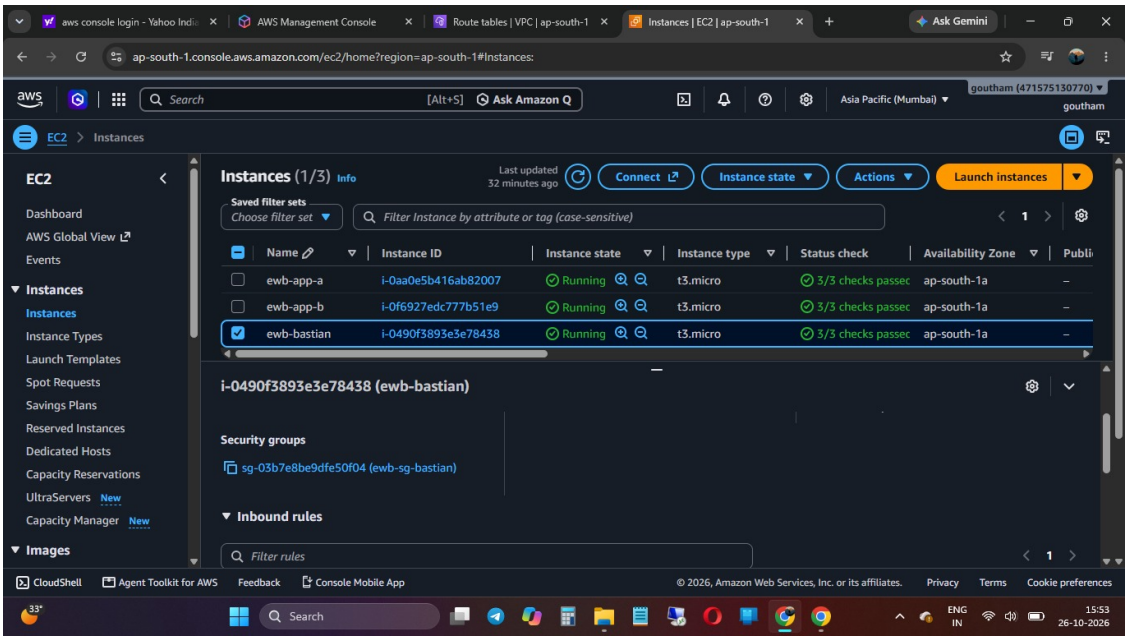


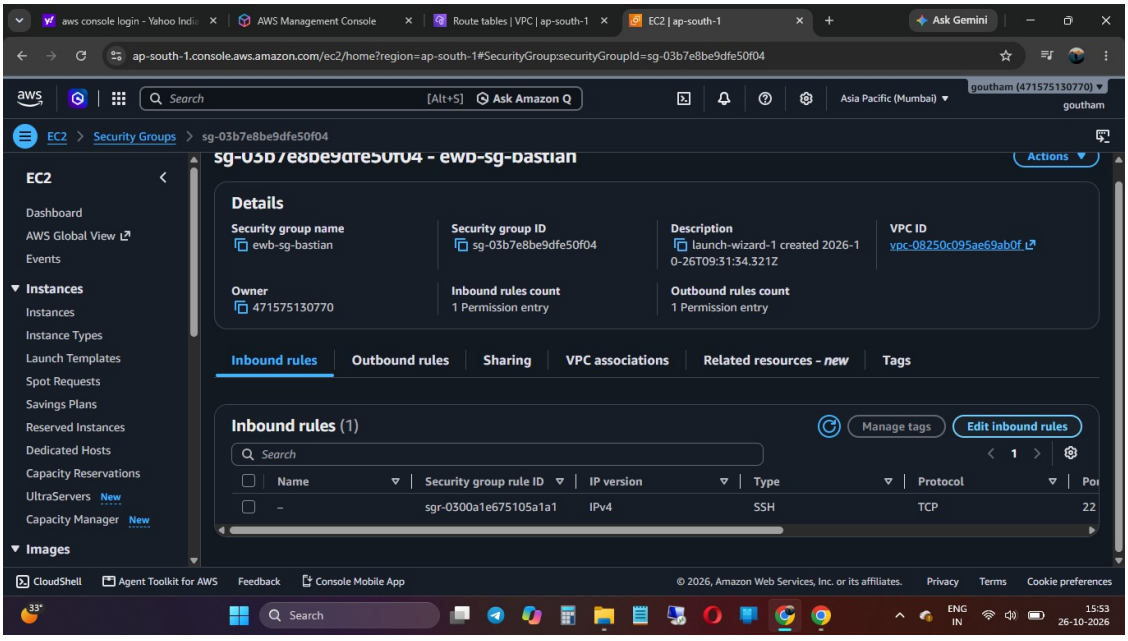
- Screenshot 9 – VPC A peering route/VPC B peering route

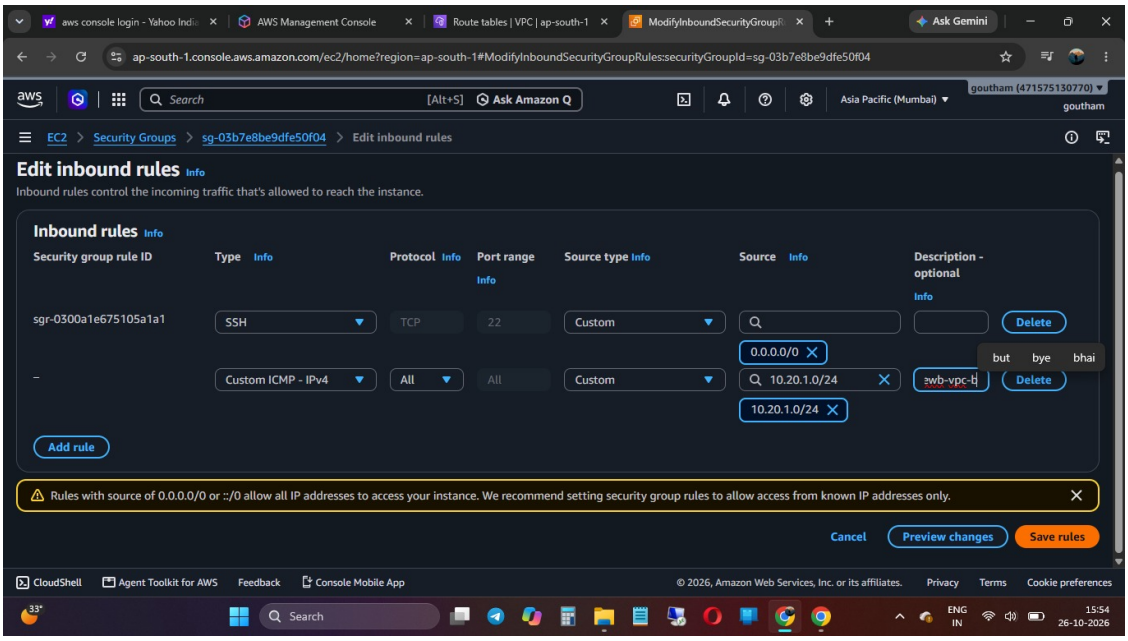


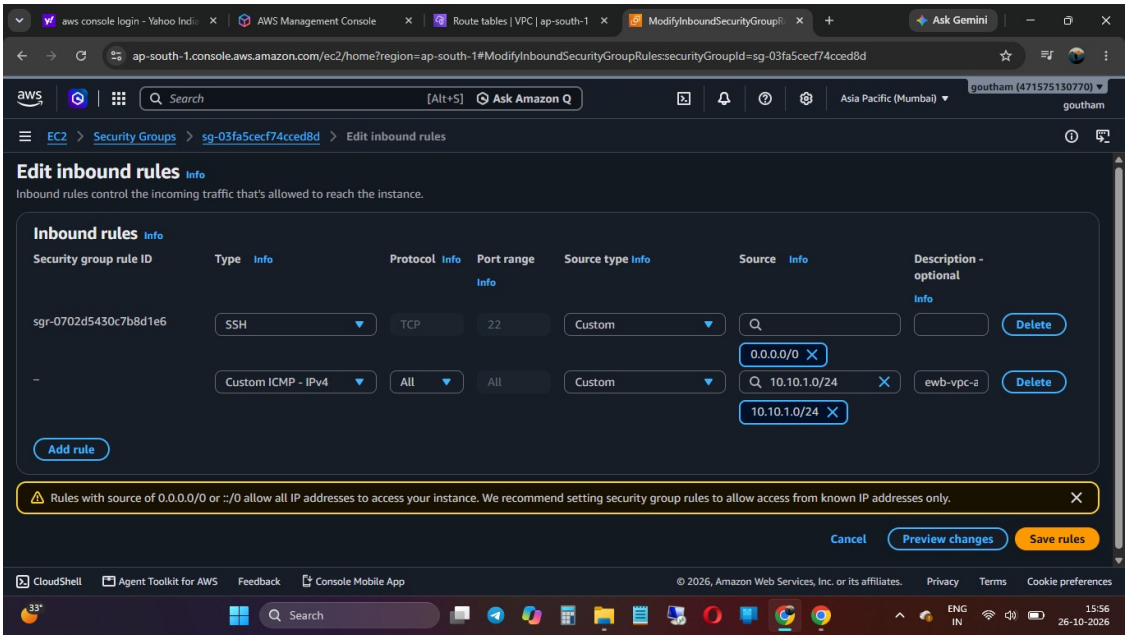


- Screenshot 10– Security group rule for ewb-app-a / Security group rule for ewb-app-b

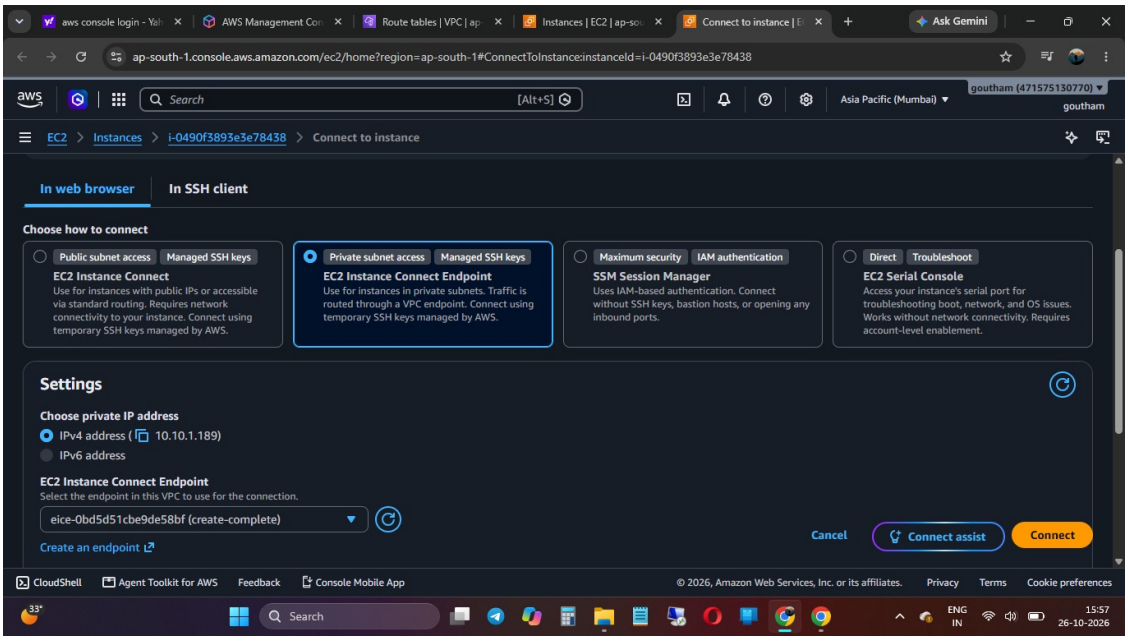

aws console login - Yahoo India
AWS Management Console
Route tables | VPC | ap-south-1
Instances | EC2 | ap-south-1
Ask Gemini
ap-south-1.console.aws.amazon.com/ec2/home?region=ap-south-1#Instances:
Search
[Alt+S]
Ask Amazon Q
Asia Pacific (Mumbai)
goutham (471575130770)
goutham
EC2 > Instances
EC2
Dashboard
AWS Global View
Events
Instances
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations
UltraServers New
Capacity Manager New
Images
Instances (1/3) Info
Last updated 32 minutes ago
Connect
Instance state
Actions
Launch instances
Saved filter sets
Choose filter set
Filter Instance by attribute or tag (case-sensitive)
Name
Instance ID
Instance state
Instance type
Status check
Availability Zone
ewb-app-a
i-0aa0e5b416ab82007
Running
t3.micro
3/3 checks passed
ap-south-1a
ewb-app-b
i-0f6927edc777b51e9
Running
t3.micro
3/3 checks passed
ap-south-1a
ewb-bastian
i-0490f3893e3e78438
Running
t3.micro
3/3 checks passed
ap-south-1a
i-0490f3893e3e78438 (ewb-bastian)
Security groups
sg-03b7e8be9dfe50f04 (ewb-sg-bastian)
Inbound rules
Filter rules
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
Search
ENG IN
15:53
26-10-2026

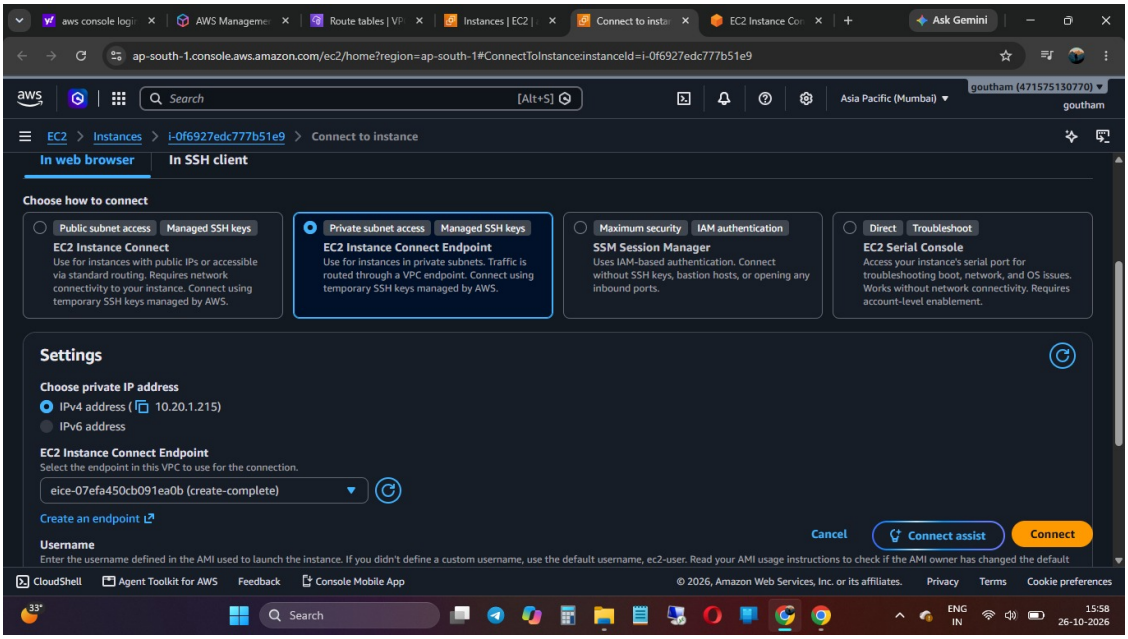

aws console login - Yahoo India
AWS Management Console
Route tables | VPC | ap-south-1
EC2 | ap-south-1
Ask Gemini
ap-south-1.console.aws.amazon.com/ec2/home?region=ap-south-1#SecurityGroup:securityGroupId=sg-03b7e8be9dfe50f04
Search
[Alt+S]
Ask Amazon Q
Asia Pacific (Mumbai)
goutham (471575130770)
goutham
EC2 > Security Groups > sg-03b7e8be9dfe50f04
EC2
Dashboard
AWS Global View
Events
Instances
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations
UltraServers New
Capacity Manager New
Images
sg-03b7e8be9dfe50f04 - ewb-sg-bastian
Actions
Details
Security group name
ewb-sg-bastian
Security group ID
sg-03b7e8be9dfe50f04
Description
launch-wizard-1 created 2026-10-26T09:31:34.321Z
VPC ID
vpc-08250c095ae69ab0f
Owner
471575130770
Inbound rules count
1 Permission entry
Outbound rules count
1 Permission entry
Inbound rules
Outbound rules
Sharing
VPC associations
Related resources - new
Tags
Inbound rules (1)
Manage tags
Edit inbound rules
Search
Name
Security group rule ID
IP version
Type
Protocol
sgr-0300a1e675105a1a1
IPv4
SSH
TCP
22
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
Search
ENG IN
15:53
26-10-2026

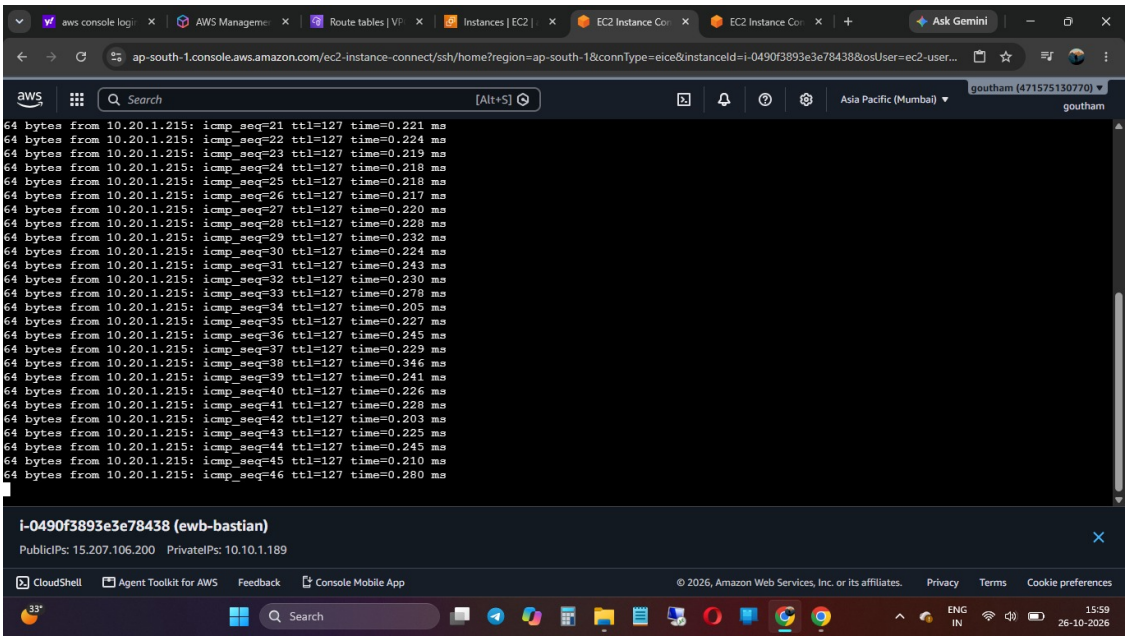


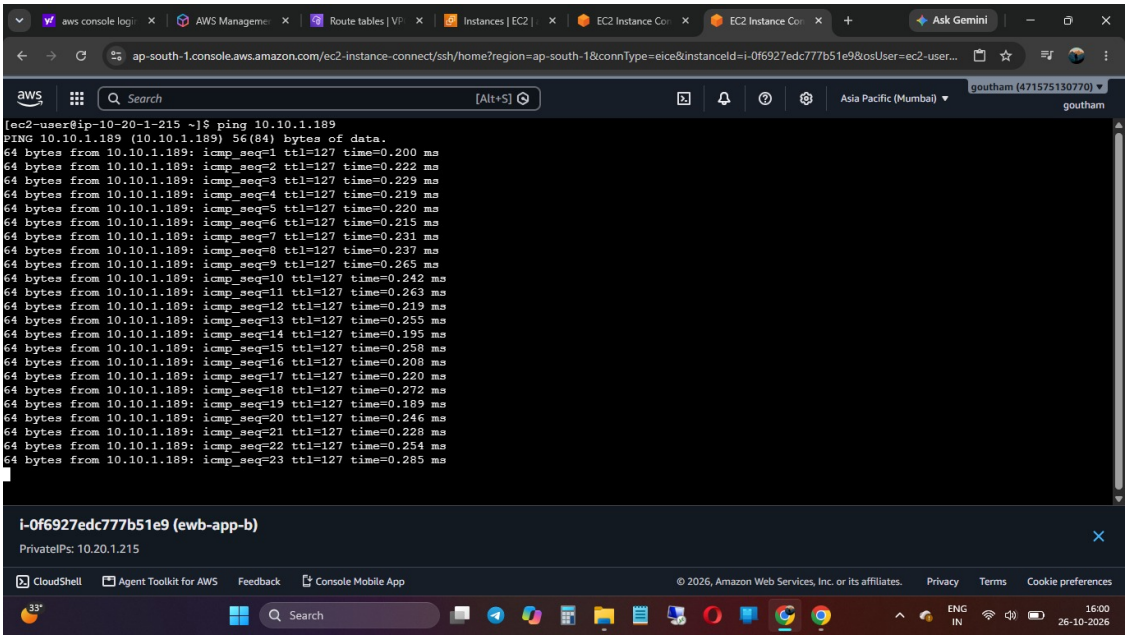


- Screenshot 11 – Successful ping from A to B / Successful ping from B to A


ap-south-1.console.aws.amazon.com/ec2/home?region=ap-south-1#ConnectToInstance:instanceId=i-0490f3893e3e78438
Search
[Alt+S]
Asia Pacific (Mumbai)
goutham (471575130770)
goutham
EC2 > Instances > i-0490f3893e3e78438 > Connect to instance
In web browser
In SSH client
Choose how to connect
Public subnet access
Managed SSH keys
EC2 Instance Connect
Use for instances with public IPs or accessible via standard routing. Requires network connectivity to your instance. Connect using temporary SSH keys managed by AWS.
Private subnet access
Managed SSH keys
EC2 Instance Connect Endpoint
Use for instances in private subnets. Traffic is routed through a VPC endpoint. Connect using temporary SSH keys managed by AWS.
Maximum security
IAM authentication
SSM Session Manager
Uses IAM-based authentication. Connect without SSH keys, bastion hosts, or opening any inbound ports.
Direct
Troubleshoot
EC2 Serial Console
Access your instance's serial port for troubleshooting boot, network, and OS issues. Works without network connectivity. Requires account-level enablement.
Settings
Choose private IP address
IPv4 address (10.10.1.189)
IPv6 address
EC2 Instance Connect Endpoint
Select the endpoint in this VPC to use for the connection.
eice-0bd5d51cbe9de58bf (create-complete)
Create an endpoint
Cancel
Connect assist
Connect
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
15:57
26-10-2026


ap-south-1.console.aws.amazon.com/ec2/home?region=ap-south-1#ConnectToInstance:instanceId=i-0f6927edc777b51e9
Search
[Alt+S]
Asia Pacific (Mumbai)
goutham (471575130770)
goutham
EC2 > Instances > i-0f6927edc777b51e9 > Connect to instance
In web browser
In SSH client
Choose how to connect
Public subnet access
Managed SSH keys
EC2 Instance Connect
Use for instances with public IPs or accessible via standard routing. Requires network connectivity to your instance. Connect using temporary SSH keys managed by AWS.
Private subnet access
Managed SSH keys
EC2 Instance Connect Endpoint
Use for instances in private subnets. Traffic is routed through a VPC endpoint. Connect using temporary SSH keys managed by AWS.
Maximum security
IAM authentication
SSM Session Manager
Uses IAM-based authentication. Connect without SSH keys, bastion hosts, or opening any inbound ports.
Direct
Troubleshoot
EC2 Serial Console
Access your instance's serial port for troubleshooting boot, network, and OS issues. Works without network connectivity. Requires account-level enablement.
Settings
Choose private IP address
IPv4 address (10.20.1.215)
IPv6 address
EC2 Instance Connect Endpoint
Select the endpoint in this VPC to use for the connection.
eice-07efa450cb091ea0b (create-complete)
Create an endpoint
Username
Enter the username defined in the AMI used to launch the instance. If you didn't define a custom username, use the default username, ec2-user. Read your AMI usage instructions to check if the AMI owner has changed the default
Cancel
Connect assist
Connect
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
15:58
26-10-2026


ap-south-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?region=ap-south-1&connType=eice&instanceId=i-0490f3893e3e78438&osUser=ec2-user...
Search
[Alt+S]
Asia Pacific (Mumbai)
goutham (471575130770)
goutham
64 bytes from 10.20.1.215: icmp_seq=21 ttl=127 time=0.221 ms
64 bytes from 10.20.1.215: icmp_seq=22 ttl=127 time=0.224 ms
64 bytes from 10.20.1.215: icmp_seq=23 ttl=127 time=0.219 ms
64 bytes from 10.20.1.215: icmp_seq=24 ttl=127 time=0.218 ms
64 bytes from 10.20.1.215: icmp_seq=25 ttl=127 time=0.218 ms
64 bytes from 10.20.1.215: icmp_seq=26 ttl=127 time=0.217 ms
64 bytes from 10.20.1.215: icmp_seq=27 ttl=127 time=0.220 ms
64 bytes from 10.20.1.215: icmp_seq=28 ttl=127 time=0.228 ms
64 bytes from 10.20.1.215: icmp_seq=29 ttl=127 time=0.232 ms
64 bytes from 10.20.1.215: icmp_seq=30 ttl=127 time=0.224 ms
64 bytes from 10.20.1.215: icmp_seq=31 ttl=127 time=0.243 ms
64 bytes from 10.20.1.215: icmp_seq=32 ttl=127 time=0.230 ms
64 bytes from 10.20.1.215: icmp_seq=33 ttl=127 time=0.278 ms
64 bytes from 10.20.1.215: icmp_seq=34 ttl=127 time=0.205 ms
64 bytes from 10.20.1.215: icmp_seq=35 ttl=127 time=0.227 ms
64 bytes from 10.20.1.215: icmp_seq=36 ttl=127 time=0.245 ms
64 bytes from 10.20.1.215: icmp_seq=37 ttl=127 time=0.229 ms
64 bytes from 10.20.1.215: icmp_seq=38 ttl=127 time=0.346 ms
64 bytes from 10.20.1.215: icmp_seq=39 ttl=127 time=0.241 ms
64 bytes from 10.20.1.215: icmp_seq=40 ttl=127 time=0.226 ms
64 bytes from 10.20.1.215: icmp_seq=41 ttl=127 time=0.228 ms
64 bytes from 10.20.1.215: icmp_seq=42 ttl=127 time=0.203 ms
64 bytes from 10.20.1.215: icmp_seq=43 ttl=127 time=0.225 ms
64 bytes from 10.20.1.215: icmp_seq=44 ttl=127 time=0.245 ms
64 bytes from 10.20.1.215: icmp_seq=45 ttl=127 time=0.210 ms
64 bytes from 10.20.1.215: icmp_seq=46 ttl=127 time=0.280 ms
i-0490f3893e3e78438 (ewb-bastian)
PublicIPs: 15.207.106.200 PrivateIPs: 10.10.1.189
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
15:59
26-10-2026


ap-south-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?region=ap-south-1&connType=eice&instanceId=i-0f6927edc777b51e9&osUser=ec2-user...
Search
[Alt+S]
Asia Pacific (Mumbai)
goutham (471575130770)
goutham
[ec2-user@ip-10-20-1-215 ~]$ ping 10.10.1.189
PING 10.10.1.189 (10.10.1.189) 56(84) bytes of data.
64 bytes from 10.10.1.189: icmp_seq=1 ttl=127 time=0.200 ms
64 bytes from 10.10.1.189: icmp_seq=2 ttl=127 time=0.222 ms
64 bytes from 10.10.1.189: icmp_seq=3 ttl=127 time=0.229 ms
64 bytes from 10.10.1.189: icmp_seq=4 ttl=127 time=0.219 ms
64 bytes from 10.10.1.189: icmp_seq=5 ttl=127 time=0.220 ms
64 bytes from 10.10.1.189: icmp_seq=6 ttl=127 time=0.215 ms
64 bytes from 10.10.1.189: icmp_seq=7 ttl=127 time=0.231 ms
64 bytes from 10.10.1.189: icmp_seq=8 ttl=127 time=0.237 ms
64 bytes from 10.10.1.189: icmp_seq=9 ttl=127 time=0.265 ms
64 bytes from 10.10.1.189: icmp_seq=10 ttl=127 time=0.242 ms
64 bytes from 10.10.1.189: icmp_seq=11 ttl=127 time=0.263 ms
64 bytes from 10.10.1.189: icmp_seq=12 ttl=127 time=0.219 ms
64 bytes from 10.10.1.189: icmp_seq=13 ttl=127 time=0.255 ms
64 bytes from 10.10.1.189: icmp_seq=14 ttl=127 time=0.195 ms
64 bytes from 10.10.1.189: icmp_seq=15 ttl=127 time=0.258 ms
64 bytes from 10.10.1.189: icmp_seq=16 ttl=127 time=0.208 ms
64 bytes from 10.10.1.189: icmp_seq=17 ttl=127 time=0.220 ms
64 bytes from 10.10.1.189: icmp_seq=18 ttl=127 time=0.272 ms
64 bytes from 10.10.1.189: icmp_seq=19 ttl=127 time=0.189 ms
64 bytes from 10.10.1.189: icmp_seq=20 ttl=127 time=0.246 ms
64 bytes from 10.10.1.189: icmp_seq=21 ttl=127 time=0.228 ms
64 bytes from 10.10.1.189: icmp_seq=22 ttl=127 time=0.254 ms
64 bytes from 10.10.1.189: icmp_seq=23 ttl=127 time=0.285 ms
i-0f6927edc777b51e9 (ewb-app-b)
PrivateIPs: 10.20.1.215
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
16:00
26-10-2026